Fenton Nonlinear Wave Suite

Theoretical formulation, solver architecture, equations, and usage

This repository computes steady, two-dimensional, periodic finite-amplitude gravity waves in finite water depth, with optional collinear current. The reference calculation is a stream-function / Fourier collocation method in the Rienecker-Fenton and Fenton numerical-wave tradition: the Fourier basis satisfies the field equation and the bed condition, while Newton iteration solves the nonlinear free-surface boundary conditions and global wave-current constraints.

The main desktop application is `fenton_gui.rs`, a complete single-file Rust 2021 implementation with a native 64-bit Windows GUI, nonlinear solver, numerical verification and text-report generator. The recommended production build uses the GNU/MinGW-w64 target in **MSYS2 UCRT64** on Windows 10/11, without Visual Studio, MSVC, third-party Rust crates or a Cargo manifest. **`fenton_gui.py` is the secondary desktop implementation**, retaining its Tkinter GUI, NumPy/optional Numba solver, validation workflow and PyInstaller build. The independent C++, browser, Android, API, VBA and standalone tools retain their individual interfaces as described below.

The central engineering calculation is the nonlinear mapping

$$L = F(H, T, d, U_c),$$

where H is wave height, T is the apparent period in the fixed frame, d is still-water depth, U_c is the imposed collinear current under the selected current convention, and L is the dynamically consistent wavelength. In Fenton's formulation, a steady wave train is fundamentally defined by three length scales, H, d and L. A period can replace wavelength only when the current or wave-speed convention is also specified, because the observed period is Doppler-shifted by current. The program therefore solves wavelength, wave speed, free-surface profile, Fourier coefficients, wave-frame flux, current variables, Bernoulli constant and integral quantities as one coupled nonlinear problem.

Theoretical point	Implementation consequence
The steady-wave model is inviscid, incompressible, irrotational and two-dimensional over a horizontal bed.	The field equation is Laplace's equation; the nonlinear difficulty is concentrated at the free surface.
The Fourier terms are chosen to satisfy Laplace's equation and the bed condition exactly.	The unknowns are solved from free-surface and global constraints, not by fitting a precomputed profile.
If T is specified instead of L, current must also be specified or assumed.	The solver contains explicit Eulerian-current and Stokes/mass-transport current closures.
The method is not a Stokes or cnoidal perturbation expansion.	Increasing N increases numerical resolution rather than changing an analytical order of wave-height expansion.
The wavelength-only and surrogate layers are secondary tools.	They should not be used when velocities, pressures, integral quantities or near-limiting behavior are required.

1. Repository components

Group	Files	Role in the toolchain
Main Windows GUI and CLI solver (Rust)	`fenton_gui.rs`	**Production single-file Rust 2021 application: Win32 GUI, optional command-line build, nonlinear Fourier solver, SVD, interval certificates and 107-character engineering report; no external Rust crates.**
Secondary Python GUI solver	`fenton_gui.py`	Python nonlinear solver, Tkinter GUI, numerical acceptance checks, interval certificates and standalone PyInstaller option.
Additional C++ GUI solver	`fenton_gui.cpp`	Windows C++ GUI with finite-system interval and output-accuracy checks.
Android app	`fenton.apk`	Fenton App for Android.
Wavelength API	`function.py`	Callable wavelength function `L(H,T,d,Uc)`.
Browser solver	`fenton.html`	Self-contained worker-based solver, accuracy checks and TXT/JSON reports.
Standalone precision solver	`fourier_precision.cpp`	Analytic Jacobian, adaptive continuation, conditioning-based Fourier order and four result/audit files.
Standalone SVD solver	`fourier_carvalho.cpp`	Analytic Jacobian, damped SVD Newton steps, adaptive continuation and three Fenton-style result files.
Tables	`tables.py`	Batch wavelength-table generation.
Pade surrogate	`pade.py`, `pade.bas`	Rational approximation model.
Neural surrogate	`neural.py`, `neural.bas`	Tanh-network approximation model.
Genetic model	`genetic.py`, `genetic.bas`	Explicit symbolic-regression wavelength formula.
Effective-variable model	`effective.bas`	Effective-period/effective-depth wavelength approximation.

Group	Files	Role in the toolchain
Nomograms	`nomogram_plots.py`, `nomogen_plots.py`	Printable design-chart generation.
Nonlinear Excel/VBA solver	`fenton.bas`	`L_fenton` wavelength function and `Fenton_report` numerical verification report, with prescribed Eulerian current.
Spreadsheet deployment	Other `*.bas` modules, `wavelenght.xlsm`	Spreadsheet implementation layer for the respective methods.

1.1 Rust application structure

`fenton_gui.rs` is self-contained and uses only the Rust standard library and native Win32 system libraries. The file contains the following logical modules:

Rust module	Responsibility
`linalg`	Dense, column-equilibrated one-sided Jacobi singular-value decomposition (SVD) for Newton corrections, with singular-value truncation and conditioning diagnostics.
`verified`	Outward-expanded binary64 intervals, finite-residual enclosures and nine engineering output-formula checks.
`solver`	Fenton stream-function/Fourier collocation, analytic Jacobian, adaptive wave-height continuation, dimensional hydrodynamics, phase extrema and numerical acceptance.
`report`	Fixed-width (107-character) report, comparison of no-current and prescribed-current scenarios, Fenton `SOLUTION.RES` tables and parameter glossary.
`windows_gui`	Native Win32 interface, UTF-16 controls, worker-thread execution and `output.txt` export.
`main`	Windows graphical entry point and conditional `headless` command-line entry point.

For nonzero prescribed current, the Rust application solves no-current and with-current cases in independent scoped threads. The Win32 interface uses a separate calculation worker so that the event loop remains responsive. It creates `output.txt` in the **process working directory** as UTF-8 text; the same result is displayed in the GUI. The values are computed from the verified nonlinear state, not from fixed demonstration tables. The implementation requests 50 Fourier harmonics and reduces the active order if numerical conditioning requires it.

The source uses IEEE-754 binary64 (`f64`) for its primary numerical calculations. `-C panic=unwind` must remain enabled so `catch_unwind` can handle exceptions raised during the finite-system verification. The method verifies finite-system expressions and engineering outputs within its stated numerical budgets; these checks do not prove existence or uniqueness of an exact solution to the continuous free-boundary problem.

The hierarchy is intentional. The stream-function / Fourier solver is the physical reference model. `fenton.bas` solves the coupled nonlinear Fourier system and checks selected integral outputs internally, even when the worksheet requests only wavelength. The separate API, approximation modules and nomograms provide their documented interfaces for wavelength estimation, tabulation or graphical design work.

---

2. Physical problem and travelling-wave formulation

The physical model is the classical steady progressive wave problem used in Fenton's papers: a homogeneous inviscid fluid, irrotational motion, incompressibility, a flat impermeable bed and a single periodic wave train. The approximation is local in the engineering sense: it is appropriate where the bed can be treated as horizontal over one wavelength and where the wave is non-breaking and changes slowly compared with the periodic motion.

A fixed frame (x, y) is used with x in the propagation direction, $y = 0$ at the still-water level, $y = -d$ at the bed, and $y = \tilde{\eta}(x, t)$ at the free surface. A wave-frame coordinate is introduced by

$$X_{\mathrm{dim}} = x - ct,$$

where c is the wave celerity in the fixed frame. In this frame the surface is steady:

$$\tilde{\eta}(x, t) = \eta(X_{\mathrm{dim}}).$$

The period, wavelength, wave number and celerity are related by

$$c = \frac{L}{T}, \qquad k = \frac{2\pi}{L}, \qquad \omega = \frac{2\pi}{T}.$$

In the program outputs, the reported celerity is the value obtained from the final solved wavelength and the specified period, so the printed geometry remains consistent with this kinematic identity.

Periodicity requires that the surface and the full flow field repeat every wavelength:

$$\eta(X_{\mathrm{dim}} + L) = \eta(X_{\mathrm{dim}}).$$

The specified wave height is the crest-to-trough difference,

$$H = \eta_{\mathrm{crest}} - \eta_{\mathrm{trough}},$$

which becomes, in the nondimensional collocation equations, a difference between the crest and trough ordinates.

Input	Meaning	Typical units
H	Crest-to-trough wave height	m
T	Apparent wave period in the fixed frame	s
d	Still-water depth	m
U_c	Collinear current under the selected convention	m/s
N	Fourier order / number of half-wave intervals	dimensionless

The current U_c is not a harmless correction added after solving a no-current wave. In a steady-wave theory the period observed in the fixed frame is affected by current, and the fixed-frame horizontal velocities require a current convention. The program therefore separates Eulerian current, Stokes/mass-transport current, mean wave-frame velocity and volume flux.

---

3. Governing equations before discretization

3.1 Potential-flow equations

The physical velocity components in the fixed frame are obtained from a velocity potential ϕ:

$$u = \frac{\partial \phi}{\partial x}, \qquad v = \frac{\partial \phi}{\partial y}.$$

Incompressibility requires zero divergence,

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0,$$

and irrotationality requires zero vorticity,

$$\frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} = 0.$$

Together they imply that the potential satisfies Laplace's equation throughout the unknown water domain:

$$\nabla^2 \phi = \frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} = 0,$$

with the vertical range

$$-d \le y \le \eta(X_{\mathrm{dim}}).$$

The bed is impermeable, so the normal velocity there is zero:

$$v(x, -d, t) = 0.$$

3.2 Free-surface kinematic condition

A fluid particle on the free surface remains on the free surface. In the fixed frame this condition is

$$\frac{\partial \eta}{\partial t} + u\frac{\partial \eta}{\partial x} = v \qquad \text{at } y = \eta(x, t).$$

In the wave frame, $U = u - c$ and $V = v$, and the free surface is steady. The same condition becomes

$$U\eta_x = V \qquad \text{on } y = \eta(X_{\text{dim}}),$$

or equivalently

$$(u - c)\eta_x = v \qquad \text{on } y = \eta(X_{\text{dim}}).$$

This is why the stream-function formulation is efficient: in the moving frame the free surface is a streamline, so the kinematic boundary condition can be enforced by making the stream function constant on the unknown surface.

3.3 Free-surface dynamic condition

The pressure is atmospheric and therefore constant on the free surface. Bernoulli's equation in the fixed frame is

$$\frac{1}{2}(u^2 + v^2) + gy + \frac{p}{\rho} + \frac{\partial \phi}{\partial t} = B(t).$$

In the frame moving with the wave, the dynamic condition on the free surface can be written as

$$\frac{1}{2}\left[(u - c)^2 + v^2\right] + g\eta = r,$$

where r uses the still-water-level datum; the corresponding bed-origin constant is $R = r + gd$. This is the main nonlinear boundary condition: velocities are squared and evaluated on the unknown free-surface elevation. The nonlinear solver must therefore find the surface and the velocity field simultaneously.

3.4 Linear dispersion as initialization only

For infinitesimal-amplitude waves without current, linear Airy theory gives

$$\omega^2 = gk \tanh(kd).$$

In wavelength form,

$$\left(\frac{2\pi}{T}\right)^2 = g\frac{2\pi}{L}\tanh\left(\frac{2\pi d}{L}\right).$$

The familiar limiting estimates are

$$L_0 \approx \frac{gT^2}{2\pi}$$

for deep water and

$$c \approx \sqrt{gd}, \qquad L \approx T\sqrt{gd}$$

for shallow water. With a collinear current, the positive intrinsic-frequency branch is

$$\omega - kU_c = \sqrt{gk\tanh(kd)},$$

or equivalently

$$\left(\frac{2\pi}{T} - kU_c\right)^2 = gk\tanh(kd),$$

with branch selection required in adverse-current or near-blocking cases. These formulas are useful for initialization, checking and wavelength-only baselines. They are not the final finite-amplitude theory used by the full solver.

---

4. Stream-function formulation

4.1 Definition and boundary interpretation

For two-dimensional incompressible flow in the wave frame, the stream function ψ is defined by

$$U = \frac{\partial\psi}{\partial y}, \qquad V = -\frac{\partial\psi}{\partial x},$$

where $U = u - c$ and $V = v$. Continuity is then satisfied identically. Irrotationality becomes

$$\nabla^2\psi = 0.$$

With the constant in ψ chosen conveniently, the impermeable bed and the moving-frame free surface are streamlines:

$$\psi(x, -d) = 0,$$
$$\psi(x, \eta(x)) = -Q,$$

where Q is the volume flux per unit width in the moving frame. The sign convention follows Fenton's steady-frame formulation: in the wave frame the mean flow is usually in the negative direction under a wave propagating in the positive fixed-frame direction.

4.2 Nondimensional wave-frame coordinates

The numerical formulation uses the wave phase and the mean-level vertical coordinate,

$$X = k(x - ct), \qquad Y = ky, \qquad k = \frac{2\pi}{L}.$$

One wavelength corresponds to

$$X \in [0, 2\pi],$$

and the bed is located at

$$Y = -kd.$$

The nondimensional free surface is written as

$$Y = \zeta(X) = k\eta(X).$$

Define the nondimensional stream function

$$\Psi = \psi\sqrt{\frac{k^3}{g}}.$$

The nondimensional wave-frame velocities are

$$\hat{U} = U\sqrt{\frac{k}{g}} = \frac{\partial\Psi}{\partial Y}, \qquad \hat{V} = V\sqrt{\frac{k}{g}} = -\frac{\partial\Psi}{\partial X}.$$

The Cauchy-Riemann relations between the nondimensional potential and stream function are

$$\Phi_X = \Psi_Y, \qquad \Phi_Y = -\Psi_X.$$

The field equation remains Laplace's equation in nondimensional form:

$$\Psi_{XX} + \Psi_{YY} = 0.$$

The bed and free-surface streamline conditions are

$$\Psi(X, -kd) = 0,$$

and

$$\Psi(X, \zeta(X)) = -Q\sqrt{\frac{k^3}{g}}.$$

The dynamic boundary condition becomes

$$\frac{1}{2}\left(\Psi_X^2 + \Psi_Y^2\right) + \zeta(X) = \frac{rk}{g},$$

or, in residual form with the same mean-level constant r,

$$\frac{1}{2}\left(\Psi_X^2 + \Psi_Y^2\right) + \zeta(X) - \frac{rk}{g} = 0.$$

The deep-water limiting case is represented by

$$kd \to \infty,$$

with the finite-depth hyperbolic functions replaced by their exponential limits.

---

5. Fourier representation and collocation

5.1 Harmonic basis satisfying the field equation

The stream-function / Fourier method is efficient because each vertical basis function is harmonic and satisfies the bottom condition before the nonlinear equations are assembled. A finite-depth nondimensional representation consistent with Fenton's 1988 formulation is

$$\Psi(X, Y) = -\bar{U}^*(Y + kd) + \sum_{j=1}^{N} B_j \frac{\sinh[j(Y + kd)]}{\cosh(jkd)} \cos(jX),$$

where

$$\bar{U}^* = \bar{U}\sqrt{\frac{k}{g}}.$$

At the bed $Y = -kd$, both $Y + kd$ and the hyperbolic sine terms vanish, so

$$\Psi(X, -kd) = 0.$$

Differentiating the series gives the nondimensional wave-frame velocity components:

$$\hat{U}(X,Y) = -\bar{U}^* + \sum_{j=1}^{N} jB_j \frac{\cosh[j(Y+kd)]}{\cosh(jkd)} \cos(jX),$$
$$\hat{V}(X,Y) = \sum_{j=1}^{N} jB_j \frac{\sinh[j(Y+kd)]}{\cosh(jkd)} \sin(jX).$$

The equivalent hyperbolic identities are

$$S_j(Y) = \frac{\sinh[j(Y+kd)]}{\cosh(jkd)} = \sinh(jY) + \cosh(jY)\tanh(jkd),$$

$$C_j(Y) = \frac{\cosh[j(Y+kd)]}{\cosh(jkd)} = \cosh(jY) + \sinh(jY)\tanh(jkd).$$

The numerical kernels evaluate these ratios with scaled exponentials. With $a = jY$ and $b = jkd$,

$$S_j(Y) = \frac{e^a - e^{-a-2b}}{1+e^{-2b}}, \qquad C_j(Y) = \frac{e^a + e^{-a-2b}}{1+e^{-2b}}.$$

This avoids forming large hyperbolic functions and subtracting nearly equal terms at the bed. In particular, $S_j(-kd) = 0$ and $C_j(-kd) = 1/\cosh(jkd)$. At fixed Y, the deep-water limit is $S_j(Y), C_j(Y) \to e^{jY}$; the finite-depth kernel retains the reflected exponential so that the bed condition remains satisfied.

5.2 Free-surface representation

A symmetric periodic wave can be represented by cosine harmonics. For the surface itself one may write

$$\eta(x) = \sum_{n=0}^{N} a_n \cos(nkx),$$

or in nondimensional phase form,

$$\zeta(X) = \sum_{n=0}^{N} a_n \cos(nX).$$

The implementation follows the Rienecker-Fenton/Fenton numerical approach more directly: it solves for the point ordinates of the free surface at collocation nodes rather than using the surface Fourier coefficients as the primary unknowns. A standard half-wave distribution is

$$x_m = \frac{m\pi}{Nk}, \qquad m = 0, 1, \ldots, N,$$

or in nondimensional form

$$X_m = \frac{m\pi}{N}, \qquad m = 0, 1, \ldots, N.$$

At every node,

$$\zeta_m = \zeta(X_m) = k\eta(X_m).$$

Using the flux convention

$$q^* = \bar{U}^* kd - Q\sqrt{\frac{k^3}{g}},$$

the kinematic free-surface condition can be written as the residual

$$R_m^{(\psi)} = \sum_{j=1}^{N} B_j S_j(\zeta_m)\cos(jX_m) - \bar{U}^*\zeta_m - q^* = 0.$$

The dynamic condition is enforced at the same nodes:

$$R_m^{(B)} = \frac{1}{2}\left[\hat{U}_m^2 + \hat{V}_m^2\right] + \zeta_m - r^* = 0,$$

where

$$r^* = \frac{rk}{g},$$

and

$$\hat{U}_m = -\bar{U}^* + \sum_{j=1}^{N} jB_j C_j(\zeta_m)\cos(jX_m),$$
$$\hat{V}_m = \sum_{j=1}^{N} jB_j S_j(\zeta_m)\sin(jX_m).$$

This separation is important when current is prescribed because the same relative wave form can correspond to different fixed-frame velocities and flux/current conventions.

---

6. Dimensionless variables and `solution.res` interpretation

The solver reports quantities using two nondimensional systems: one based on wave number k, and one based on water depth d. The k-based form is natural for Fourier theory, while the d-based form is convenient for engineering interpretation and comparison between depths.

The core nondimensional engineering groups are

$$\hat{H} = \frac{H}{d}, \qquad \hat{L} = \frac{L}{d}, \qquad \hat{c} = \frac{c}{\sqrt{gd}}, \qquad \hat{U}_c = \frac{U_c}{\sqrt{gd}}.$$

Depth and steepness are interpreted using

$$\frac{d}{L}, \qquad \frac{H}{L}, \qquad kd = \frac{2\pi d}{L}.$$

The two common dimensionless periods are

$$\tau_d = T\sqrt{\frac{g}{d}}, \qquad \tau_k = T\sqrt{gk} = \tau_d\sqrt{kd}.$$

The associated depth-based frequency variables are

$$\sigma_d = \frac{2\pi}{\tau_d}, \qquad \alpha_d = \sigma_d^2.$$

The Ursell number, current Froude number and Doppler parameter used for regime interpretation are

$$Ur = \frac{HL^2}{d^3},$$

$$Fr_c = \frac{U_c}{\sqrt{gd}},$$

and

$$D = \frac{U_c T}{L}.$$

The intrinsic wave-current frequency is

$$\omega - kU_c = \omega_{\text{intrinsic}}(k).$$

These nondimensional groups explain why the finite-amplitude wavelength is generally not the linear Airy wavelength:

$$L \neq L_{\text{Linear-Airy}}.$$

6.1 Dimensionless-variable table

Quantity	k-based nondimensional value	d-based nondimensional value
Water depth	kd	$d/d = 1$
Wave length	$k\lambda = 2\pi$	λ/d
Wave height	kH	H/d
Wave period	$T\sqrt{gk}$	$T\sqrt{g/d}$
Wave speed	$c\sqrt{k/g}$	$c/\sqrt{gd}$
Eulerian current	$\bar{u}_1\sqrt{k/g}$	$\bar{u}_1/\sqrt{gd}$
Stokes current / mass-transport current	$\bar{u}_2\sqrt{k/g}$	$\bar{u}_2/\sqrt{gd}$

Quantity	k-based nondimensional value	d-based nondimensional value
Mean wave-frame speed	$\bar{U}\sqrt{k/g}$	$\bar{U}/\sqrt{gd}$
Wave volume flux	$q\sqrt{k^3/g}$	$q/\sqrt{gd^3}$
Bernoulli offset	rk/g	r/gd
Volume flux	$Q\sqrt{k^3/g}$	$Q/\sqrt{gd^3}$
Bernoulli constant	Rk/g	R/gd
Momentum flux	$Sk^2/(\rho g)$	$S/(\rho g d^2)$
Impulse	$I\sqrt{k^3}/(\rho\sqrt{g})$	$I/(\rho\sqrt{gd^3})$
Kinetic energy	$T_k k^2/(\rho g)$	$T_k/(\rho g d^2)$
Potential energy	$V_p k^2/(\rho g)$	$V_p/(\rho g d^2)$
Mean square orbital bed velocity	$\langle(u_b-\bar{u}_1)^2\rangle k/g$	$\langle(u_b-\bar{u}_1)^2\rangle/(gd)$
Radiation stress	$S_{xx}k^2/(\rho g)$	$S_{xx}/(\rho g d^2)$
Wave power	$Fk^{5/2}/(\rho g^{3/2})$	$F/(\rho g^{3/2}d^{5/2})$

The Fourier coefficient table is separate: its two columns contain the native stream-function coefficients B_j and surface coefficients E_j, not two alternative scalings of one quantity. The surface reconstruction is

$$\zeta(X) = \sum_{j=1}^{N-1} E_j \cos(jX) + \frac{E_N}{2}\cos(NX).$$

Thus the terminal cosine amplitude is $E_N/2$. The dimensional surface elevation is $\eta = \zeta/k$; the bed-based ordinate in the standalone surface and flow files is $y/d = 1 + \zeta/(kd)$.

The current definitions used by Fenton can be summarized as

$$\bar{u}_1 = c - \bar{U},$$

and

$$\bar{u}_2 = c - \frac{Q}{d}.$$

The flux variable used in the nonlinear equations is

$$q = \bar{U}d - Q,$$

and the Bernoulli offset is commonly written as

$$r = R - gd$$

when the Bernoulli constant is shifted from a bed-origin formulation to a mean-level formulation.

7. The z[i] state vector and solver description

7.1 Physical meaning of the z[i] equations

The z[i] equations are not empirical curve-fitting equations. They are the algebraic, nondimensional form of the steady nonlinear free-boundary problem. Solving them means finding a single state vector that simultaneously satisfies:

1. the prescribed wave height and water depth,
2. the specified wavelength or period relation,
3. the selected Eulerian-current or Stokes-current closure,
4. the mean-level convention,
5. the crest-to-trough wave-height condition,
6. the free-surface streamline condition at every collocation node,
7. Bernoulli's equation at every collocation node, and
8. the Fourier representation of a harmonic velocity field.

The unknown vector can be understood as

$$\mathrm{z} = [\text{global scalars} \mid \text{surface ordinates} \mid \text{Fourier coefficients}].$$

The primary physical correspondence in the Fenton-style 1-based state-vector convention is

$$\begin{aligned} z_1 &\leftrightarrow kd, & z_2 &\leftrightarrow kH, & z_3 &\leftrightarrow T\sqrt{gk}, \\ z_4 &\leftrightarrow c\sqrt{\frac{k}{g}}, & z_5 &\leftrightarrow \bar{u}_1\sqrt{\frac{k}{g}}, & z_6 &\leftrightarrow \bar{u}_2\sqrt{\frac{k}{g}}, \\ z_7 &\leftrightarrow \bar{U}\sqrt{\frac{k}{g}}, & z_8 &\leftrightarrow q\sqrt{\frac{k^3}{g}}, & z_9 &\leftrightarrow \frac{rk}{g}. \end{aligned}$$

Block	Indices	Physical/numerical content
Global scalars	z[1]...z[9]	kd, kH, period/celerity variables, current variables, flux variable and Bernoulli offset.
Free surface	z[10]...z[N+10]	Nodal ordinates $k\eta_m$ from crest to trough.
Fourier modes	z[N+11]...z[2N+10]	Harmonic stream-function coefficients B_j.

At the requested order $N = 50$, this indexing corresponds to 110 active 1-based state entries in the Fenton-style indexing convention used by the code. Some ports pack only the independent optimization variables into a shorter zero-based vector; the physical meaning remains the same.

7.2 Current definitions inside the state vector

The program explicitly distinguishes Eulerian current, Stokes/mass-transport current, wave-frame mean velocity and flux. The Eulerian-current variable is connected to celerity and mean wave-frame velocity by

$$\bar{u}_1 = c - \bar{U}.$$

The Stokes-current or mass-transport current is connected to volume flux by

$$\bar{u}_2 = c - \frac{Q}{d}.$$

The wave volume flux convention is

$$q = \bar{U}d - Q.$$

This is not a cosmetic distinction. Prescribing Eulerian current and prescribing Stokes current close different mathematical problems. The residual system therefore selects the active current equation through a current-criterion index.

7.3 Residual equations

The residual vector is the numerical form of the complete nonlinear wave problem. At convergence, every component of

$$\mathrm{F(z)} = 0$$

must vanish. These equations are not optional diagnostics: they are the constraints that define the solved wave. The first eight residuals impose the global wave, current, flux, mean-level and height constraints. The next two blocks are collocation equations imposed at every free-surface node.

For Fourier order N, the half-wave grid contains $N + 1$ collocation nodes,

$$X_m = \frac{m\pi}{N}, \qquad m = 0, 1, \ldots, N.$$

The complete residual system therefore contains

$$8 + (N + 1) + (N + 1) = 2N + 10$$

equations. The requested order $N = 50$ gives 110 equations. If conditioning requires a lower order, the active state and residual system are rebuilt for that order; for example, $N = 32$ gives 74 equations.

Residual group	Number of equations	Unknowns constrained	Physical purpose
Global scalar equations	8	z_1 to z_9 and current selector	Enforce depth, height, period or wavelength, celerity, current, flux, datum and crest-trough height.
Streamline collocation equations	$N+1$	Surface ordinates and Fourier coefficients	Force every free-surface node to lie on one moving-frame streamline.
Bernoulli collocation equations	$N+1$	Surface ordinates, velocities and Bernoulli offset	Force constant total head along the unknown free surface.
Complete system	$2N+10$	Full state vector z	Defines the nonlinear free-boundary wave solution.

The residuals evaluated by the implementation are

$$\begin{aligned}
r_1 &= z_2 - z_1(H/d), \\
r_2 &= z_2 - H_s z_3^2, \\
r_3 &= z_4 z_3 - 2\pi, \\
r_4 &= z_5 + z_7 - z_4, \\
r_5 &= z_1(z_6 + z_7 - z_4) - z_8, \\
r_6 &= z_{c+4} - \frac{U_c}{\sqrt{gd}}\sqrt{z_1}, \\
r_7 &= z_{10} + z_{N+10} + 2\sum_{i=1}^{N-1} z_{10+i}, \\
r_8 &= z_{10} - z_{N+10} - z_2, \\
r_{9+m} &= \psi_m - z_8 - z_7 z_{10+m}, \\
r_{N+10+m} &= \frac{1}{2}\left[(-z_7 + u_m)^2 + v_m^2\right] + z_{10+m} - z_9,
\end{aligned}$$

where $m = 0, 1, \ldots, N$ in the two collocation blocks. These expressions describe a finite-depth, period-specified case. With wavelength input, the second residual is instead

$$r_2 = z_2 - 2\pi H_s, \qquad H_s = H/L$$

at the current continuation height. The current selector in r_6 is 1 for Eulerian current and 2 for mass-transport current.

7.3.1 Notation used in the residual equations

Symbol	Meaning in the residual system	Practical interpretation
m	Collocation-node index, $m = 0, 1, \ldots, N$	Moves from crest to trough over the symmetric half-wave.
H/d	User-specified relative height	Dimensional input height divided by still-water depth.
H_s	Continuation-step value of the period-form height parameter	For a period-specified problem, it represents the current step of $H/(gT^2)$, so $H_s z_3^2 = kH$ at convergence for that step.
z_{10+m}	Free-surface ordinate at node m	Nondimensional surface elevation $k\eta_m$ solved directly by Newton iteration.
ψ_m	Fourier stream-function contribution evaluated at node m	The sum $\sum B_j S_j(\zeta_m)\cos(jX_m)$, excluding the mean-flow term.
u_m, v_m	Fourier velocity contributions at node m	Used with $-z_7$ to form the total wave-frame velocities in Bernoulli's equation.
z_7	Mean wave-frame velocity variable	Appears in both streamline and dynamic boundary conditions.
z_8	Flux-related wave-frame constant	Couples the streamline level to the volume-flux convention.
z_9	Bernoulli offset	The scalar total-head value, after datum shift, that must be the same at every surface node.
z_{c+4}	Current-selected state variable	Selects the active current definition. The selector c is not the wave celerity.

7.3.2 Global residual interpretation The residual equations are listed above in the same order in which they are solved. The table below explains the role of each global residual without changing the formula notation.

Residual	Constraint imposed	What zero residual means
r_1	Height-depth consistency	Since $z_1 = kd$ and $z_2 = kH$, the condition $r_1 = 0$ imposes $kH = kd(H/d)$. The solved nondimensional geometry therefore respects the specified physical H and d.

Residual	Constraint imposed	What zero residual means
r_2	Period-form height closure	The current continuation value of $H/(gT^2)$ is consistent with kH and $T\sqrt{gk}$. This is the period-specified counterpart of directly prescribing H/L.
r_3	Period/celerity closure	Since $z_3 = T\sqrt{gk}$ and $z_4 = c\sqrt{k/g}$, $r_3 = 0$ imposes $ckT = 2\pi$, equivalent to $c = L/T$.
r_4	Eulerian-current identity	The state variables satisfy $\bar{u}_1 = c - \bar{U}$.
r_5	Stokes-current and flux identity	The mass-transport current, mean-flow variable and flux-related constant remain mutually consistent. The flux convention cannot drift independently of the velocity field.
r_6	Prescribed-current equation	The user-specified current is imposed on the selected current variable. The selector c in z_{c+4} chooses the active current criterion and is not the wave celerity.
r_7	Mean-level convention	The discrete free surface has the required zero-mean datum. This removes the otherwise arbitrary vertical translation of the surface.
r_8	Crest-trough wave height	The difference between crest ordinate and trough ordinate equals the solved nondimensional height kH. The solver cannot converge to a nearby wave with the wrong height.

7.3.3 Collocation residual interpretation The two collocation residual families are applied at every node $m = 0, 1, \ldots, N$.

Residual family	Number of equations	Boundary condition represented	What zero residual means
r_{9+m}	$N+1$	Kinematic free-surface condition	The unknown free surface is a streamline in the moving frame. No fluid crosses the solved surface.
r_{N+10+m}	$N+1$	Dynamic free-surface condition	Enforce constant Bernoulli head and atmospheric pressure at the free-surface nodes; between-node accuracy is checked separately.

The collocation residuals are the part of the system that makes the solution fully nonlinear. The surface ordinates are unknown, and the velocities are evaluated on that unknown surface. Newton iteration therefore updates the surface profile, the Fourier coefficients, the celerity/current variables and the Bernoulli offset simultaneously.

Solver consequence	Reason
The wavelength is not obtained from linear dispersion alone.	The period closure is solved together with finite-amplitude free-surface and Bernoulli constraints.
The current affects the nonlinear solution, not only the final reported celerity.	Current variables enter the global residuals and the wave-frame velocity in the boundary conditions.
The Fourier coefficients are not fitted after the surface is known.	They are solved inside the same nonlinear system that determines the surface ordinates.
The mean water level is fixed by the residual system.	The datum equation $r_7 = 0$ removes vertical translation freedom.
The final wave height is enforced directly.	The crest-trough equation $r_8 = 0$ prevents convergence to a nearby wave with the wrong height.

Once $z_1 = kd$ has converged, the wavelength follows directly:

$$kd = \frac{2\pi d}{\lambda}.$$

Thus

$$\frac{\lambda}{d} = \frac{2\pi}{kd}, \qquad L = \lambda, \qquad c = \frac{L}{T}.$$

The residual variable z_4 enforces the same period-celerity closure during Newton iteration; final reports and API diagnostics should still derive the dimensional celerity from the final L/T relation.

7.4 Newton iteration and stabilized linear solve

The residual system is written compactly as

$$\mathrm{F}(\mathrm{z}) = 0.$$

Newton iteration linearizes the residuals about the current iterate:

$$\begin{gathered}\mathrm{J}(\mathrm{z}^{(m)})\,\Delta\mathrm{z}^{(m)} = -\mathrm{F}(\mathrm{z}^{(m)}),\\ \mathrm{z}^{(m+1)} = \mathrm{z}^{(m)} + \Delta\mathrm{z}^{(m)}.\end{gathered}$$

The Jacobian entries are

$$\mathrm{J}_{ij} = \frac{\partial F_i}{\partial z_j}.$$

The maintained GUI, browser and standalone solvers assemble analytic derivatives of the global constraints and free-surface equations. The derivatives use the same scaled-exponential basis as the residual evaluation.

The most sensitive entries involve derivatives of the streamline and Bernoulli residuals with respect to Fourier coefficients, surface ordinates, kd, celerity variables and the Bernoulli offset. Typical derivative families are

$$\frac{\partial R_m^{(\psi)}}{\partial B_n}, \qquad \frac{\partial R_m^{(\psi)}}{\partial \eta_j}, \qquad \frac{\partial R_m^{(\psi)}}{\partial k},$$

and

$$\frac{\partial R_m^{(B)}}{\partial B_n}, \qquad \frac{\partial R_m^{(B)}}{\partial \eta_j}, \qquad \frac{\partial R_m^{(B)}}{\partial r}, \qquad \frac{\partial R_m^{(B)}}{\partial c}.$$

For a dimensional Bernoulli residual,

$$R_m^{(B)} = \frac{1}{2}\left[(u_m - c)^2 + v_m^2\right] + g\eta_m - r,$$

a representative chain-rule derivative is

$$\frac{\partial R_m^{(B)}}{\partial z_j} = (u_m - c)\left(\frac{\partial u_m}{\partial z_j} - \frac{\partial c}{\partial z_j}\right) + v_m\frac{\partial v_m}{\partial z_j} + g\frac{\partial \eta_m}{\partial z_j} - \frac{\partial r}{\partial z_j}.$$

Because near-limiting waves and adverse-current cases can make the Jacobian ill-conditioned, the solvers use stabilized linear algebra. When singular value decomposition is used,

$$\mathrm{J} = \mathrm{U}\Sigma\mathrm{V}^T,$$

and the pseudoinverse Newton correction is

$$\Delta\mathrm{z} = -\mathrm{V}\Sigma^{-1}\mathrm{U}^T\mathrm{F}.$$

The numerical implementations share the governing equations but use different linear and globalization strategies:

Implementation	Linear solve and nonlinear step control
`fourier_precision.cpp`	Row/column equilibration, LU with iterative refinement, pivoted QR and regularized least-squares fallbacks; trust-region dogleg steps and positive-variable retraction.
`fourier_carvalho.cpp`	Column-equilibrated SVD, relative singular-value cutoff 10^{-15} and residual-decreasing backtracking.
`fenton_gui.rs` **(main application)**	**Analytic Jacobian, column-equilibrated one-sided Jacobi SVD,** 10^{-15} **relative singular-value threshold, residual-decreasing Newton backtracking, adaptive height continuation and interval-based output checks.**
`fenton_gui.py`, `fenton_gui.cpp`, `fenton.html`	Analytic Jacobian, scaled SVD-based correction and safeguarded adaptive continuation.
`fenton.bas`	Analytic Jacobian, column-equilibrated SVD, backtracking and adaptive height continuation.

Continuation increases wave height from a small-amplitude seed. A failed trial restores the last accepted state and reduces the height increment. Convergence is assessed at the requested final height, rather than at an intermediate wave reached before failure.

7.5 Fourier order and short-period robustness

The requested Fourier order is an upper resolution target. The solvers assess the condition number of the analytic Jacobian after column equilibration:

$$s_j = \max_i |J_{ij}|, \qquad \widetilde{J}_{ij} = \frac{J_{ij}}{s_j}, \qquad \kappa_2(\tilde{\mathrm{J}}) = \frac{\sigma_{\max}}{\sigma_{\min}}.$$

A zero column or singular system is treated as ill-conditioned. If the condition exceeds 10^{11}, the calculation restarts at a lower Fourier order. Starting from 50 modes, the available sequence is

$$50 \to 40 \to 32 \to 25 \to 20 \to 16 \to 12.$$

Each restart solves the original target wave again with the active state-vector size $2N + 10$. It does not reduce the requested physical wave height or relax the final residual tolerance. `fourier.cpp` starts at its user-requested order and uses $N_{\text{next}} = \max(12, \lfloor 0.8N \rfloor)$ when the requested order is at least 12; a smaller requested order is not increased.

Short periods can make high Fourier orders poorly conditioned for otherwise solvable waves. Adaptive height continuation, scaled basis functions and conditioning-based order selection address this numerical difficulty. A shorter period can also make the specified wave approach or exceed the limiting steady-wave height. Non-convergence is reported if the requested target cannot be verified; an intermediate-height solution is not presented as the requested result.

7.6 Convergence, conditioning and spectral resolution

Residual norms refer to the implemented nondimensional equations, with $M = 2N + 10$:

$$F_{\text{RMS}} = \sqrt{\frac{1}{M}\sum_{i=1}^{M} F_i^2}, \qquad F_{\max} = \max_i |F_i|.$$

Implementation	Intermediate residual limits	Final residual limits
`fourier_precision.cpp`	$F_{\text{RMS}} \le 10^{-10}$	$F_{\text{RMS}} \le 10^{-12}$
`fourier_carvalho.cpp`	$F_{\max} \le 10^{-10}$	$F_{\max} \le 10^{-10}$
Rust GUI/CLI (`fenton_gui.rs`)	$F_{\max} \le 10^{-8}$, together with the mean free-surface correction check	$F_{\max} \le 10^{-10}$, rechecked after dimensional normalization, with independent numerical and binary64 interval checks
Python/C++ GUI and HTML solvers	$F_{\max} \le 10^{-8}$, together with their step checks	$F_{\max} \le 10^{-10}$, rechecked after post-processing and enclosed independently
`fenton.bas`	$F_{\max} \le 10^{-10}$	$F_{\max} \le 10^{-10}$ after dimensional normalization, plus independent interval checks

The accepted state must also satisfy the finite-value and admissibility checks and the 10^{11} Jacobian-conditioning limit. These are numerical acceptance criteria for a finite Fourier system, not a proof of exact agreement with the continuous free-boundary problem.

The precision driver writes independent diagnostics to `solver.res`:

Diagnostic	Interpretation
Requested and accepted Fourier order	Identifies the resolution actually used.
Fourier order attempts	Lists each attempted order, its acceptance status and condition number.

Diagnostic	Interpretation
Adaptive continuation history	Records accepted/rejected height trials for the final order. Earlier order attempts are summarized separately.
Complete state and residual vectors	Provides all $2N + 10$ unknowns and residuals for inspection.
Overall nonlinear convergence	Requires an independently verified final state at the requested height.
Spectral resolution check	Compares the maximum between-node boundary defect with 10^{-9}.

For the precision driver's spectral check, the cosine surface supplies the initial ordinate at each off-grid phase. A safeguarded correction locates the free-surface streamline before the Bernoulli defect is evaluated. The surface output file instead samples the cosine reconstruction directly, so its pressure check need not equal the off-grid diagnostic in `solver.res`.

For the standalone drivers described above, a spectral `FAIL` is a separate resolution warning: nonlinear convergence may still be `PASS`. Both statuses must be considered when assessing a result. A failed nonlinear or conditioning check prevents the standalone solvers from writing a new result set; existing files from an earlier run may remain on disk.

7.7 Acceptance of the finite Fourier system and selected outputs

`fenton_gui.rs`, `fenton_gui.py`, `fenton_gui.cpp`, `fenton.html` and `fenton.bas` apply an output-accuracy gate after the nonlinear solve. A result is accepted only when the requested target state satisfies the conditioning, finite-value, residual, spatial-resolution and output checks. These requirements supplement the nonlinear residual limits in §7.6.

Check	Evaluation	Acceptance requirement
Finite-system residual	Independent interval evaluation of all $2N + 10$ equations at the stored state, reconstructing the dimensionless groups from the physical inputs.	Every residual interval lies inside $[-10^{-10}, 10^{-10}]$.
Interpolated free surface	Both boundary residuals evaluated on the cosine surface at $\max(2048, 32N)$ uniformly spaced phases over one wavelength.	Maximum sampled nondimensional residual no greater than 10^{-6}.
Spectral bed-velocity identities	Compare power and radiation stress with expressions using the independently evaluated absolute mean-square bed velocity.	Each difference fits the output budget below.

Check	Evaluation	Acceptance requirement
Defining integrals	Integrate dimensional velocity and Bernoulli pressure to obtain impulse, kinetic energy, potential energy, radiation stress and power.	Each reported quantity agrees with the finer quadrature within its output budget.
Quadrature refinement	Compare $\max(256, 8N)$ phases and 64 Gauss-Legendre depth points with $\max(512, 16N)$ phases and 96 depth points.	The difference between the two quadratures fits the same output budget.
Selected-output enclosures	Independently enclose nine finite-formula outputs at the stored coefficients.	The worst-case distance from the reported value to its enclosure fits a conservative output budget.

For a dimensional reference quantity q, the comparison budget is

$$\varepsilon(q) = 10^{-7} + 10^{-8}|q|.$$

The absolute term is expressed in the SI units of the quantity being compared. The residual limits are nondimensional. These values are explicit implementation acceptance choices, rather than tolerances prescribed by Fenton for every application.

If the reference is enclosed in $[q_-, q_+]$, the interval check uses

$$e_{\max} = \max(|q_{\text{reported}} - q_-|, |q_{\text{reported}} - q_+|), \qquad e_{\max} \le 10^{-7} + 10^{-8} \min_{q \in [q_-, q_+]} |q|,$$

with outward rounding for the error bound and downward rounding for the budget. The minimum magnitude is zero when the interval contains zero.

The nine enclosed outputs are **wavelength, celerity, impulse, kinetic energy, potential energy, total energy density, radiation stress, wave power and mean-square orbital bed velocity**. These enclosures describe formula evaluation at the stored finite Fourier state. The surface scan and quadrature-refinement comparisons provide additional numerical resolution checks; they are not rigorous bounds over every phase or on quadrature truncation error.

Implementation	Independent enclosure arithmetic
Rust	**Outward-expanded IEEE binary64 intervals, explicit exponential/trigonometric Taylor remainders and finite-formula checks for nine outputs.**
Python	50-digit outward-rounded `Decimal` intervals; the binary64 inputs are interpreted exactly.

Implementation	Independent enclosure arithmetic
C++	Outward-rounded IEEE binary64 intervals using adjacent representable endpoints.
HTML/JavaScript	Outward-rounded IEEE binary64 intervals with explicitly constructed adjacent endpoints.
VBA	Binary64 interval endpoints enlarged by conservative rounding margins, including a margin for subnormal results.

The interval evaluators use bounded trigonometric series independently of the ordinary collocation sine/cosine evaluations. The C++, JavaScript and VBA exponential enclosures use Taylor bounds and range reduction; their square-root endpoint checks validate the initial root estimates.

In the Rust, Python, C++ and JavaScript solver objects, successful acceptance sets `converged`, `accuracy_verified` and `finite_system_verified`. The `accuracy_metrics` object records the sampled surface residual and comparison ratios; a ratio at or below one satisfies its budget. `finite_system_certificate` records the residual and output enclosures, budgets and their scope.

`solution_status`	Meaning
`not_solved`	No completed calculation is available.
`verified_numerically`	The nonlinear solution and all implemented acceptance checks passed.
`nonconvergence`	The requested target was not obtained within the solver's admissibility, convergence or conditioning requirements.
`resolution_failure`	The collocation solve completed, but one or more resolution or output-accuracy budgets were exceeded.
`accuracy_failure`	The verification calculation could not be completed, for example because of a numerical exception.

`finite_system_verified` alone does not imply overall acceptance. A finite state can have very small collocation residuals while failing the between-node or integral checks. The Rust, Python and C++ GUIs and browser display engineering values only for accepted cases, assessing the no-current and imposed-current solutions separately. VBA returns an `Err:` message instead of a wavelength when a check fails; `Fenton_report` exposes the reason and available diagnostics.

7.8 Verification evidence and Lean scope

The Python solver computes accurately for most configurations in the benchmark matrix: 19 of 21 physical cases satisfy all acceptance requirements on both NumPy and Numba. These cases include the default wave, several small period waves, opposing current and both

current conventions in the solver core. This is evidence for the tested configurations, not a statistical success rate for arbitrary inputs.

Implementation tested	Benchmark coverage	Result
Python, NumPy and Numba	21 physical cases on each backend; 42 runs.	38 accepted runs covering 19 cases; four accuracy rejections covering two cases.
Native C++ solver core	The same 21 physical cases.	Acceptance decisions and active orders match Python; 21 compared outputs meet the comparison budget for all 19 accepted cases.
JavaScript solver under Node.js	The same 21 physical cases.	Acceptance decisions and active orders match Python; 21 compared outputs meet the comparison budget for all 19 accepted cases.
VBA through a mechanical C++ translation	18 Eulerian-current cases.	Acceptance decisions and active orders match Python; nine compared outputs meet the comparison budget for all 16 accepted cases. Native Excel/VBA execution remains untested.

Independent 80-digit calculations confirm the recorded finite-residual and nine selected-output enclosures for each implementation's tested states. The C++, JavaScript and translated VBA interval arithmetic also pass 911 arithmetic checks. Solver-core verification is distinct from testing every native GUI, spreadsheet host or browser environment.

Two steep-wave configurations are deliberately rejected by the accuracy gate:

H (m)	T (s)	d (m)	Eulerian U_c (m/s)	Active N	Sampled surface residual, approximately
2	3	5	0	32	1.18×10^{-5}
3	4	5	1	40	7.82×10^{-5}

Their collocation residuals are small, but the reconstructed surface and integral comparisons exceed the accuracy budgets. Rejection identifies insufficient numerical accuracy at the available resolution; it does not by itself establish that the physical wave cannot exist.

The companion Python verification package contains five Lean modules:

Module	Formal scope	Theorems
`FentonProduction.lean`	Governing formula, differential and scaling identities.	62
`ProductionAcceptance.lean`	Acceptance rules and contracts for evaluated maxima.	13
`FiniteSystem.lean`	Enclosure arithmetic, pi bounds and finite residual/output-budget implications.	29
`NumericalCertificates.lean`	Exact rational checks on recorded numerical observations.	314
`FiniteCertificates.lean`	Exact integer checks of recorded residual-enclosure and output-budget inequalities.	420
Total	**All modules compiled successfully.**	**838**

The Lean evidence is tied to the source snapshots and recorded states identified by the verification package. Lean checks the mathematical statements, enclosure implications and recorded inequalities. The source-to-model correspondence and runtime interval construction are reviewed and numerically tested; the complete Python, C++, JavaScript and VBA programs are not extracted from Lean or certified as executable programs. The evidence also does not prove existence or uniqueness of an exact nonlinear root, or a continuous water-wave truncation-error bound. This scope is compatible with the demonstrated accuracy of the accepted benchmark calculations.

The verification results in this section apply to the named implementations and their recorded tests. They do not certify the Android binary, separate standalone drivers, wavelength API or surrogate models. The Rust source includes its own unit and ignored numerical reference tests (see §11.1), but the preceding Python/C++/JavaScript/VBA benchmark counts and Lean certificates must not be interpreted as an independently executed validation of a particular Rust binary or compiler configuration.

---

8. Post-processing and engineering outputs

After successful numerical acceptance, the GUI and browser report dimensional engineering quantities. The standalone solvers retain their documented nondimensional output scalings and separate diagnostics. The fundamental dimensional conversions are

$$k = \frac{2\pi}{L}, \qquad c = \frac{L}{T}.$$

This post-processing convention avoids treating wavelength and celerity as independent reported quantities in period-specified runs.

The moving-frame volume flux Q and the wave flux variable q are related by

$$q = \bar{U} d - Q.$$

The mean free-surface level is

$$\bar{\eta} = \frac{1}{L} \int_0^L \eta(x)\, dx.$$

In the usual mean-level convention,

$$\bar{\eta} = 0.$$

The energy calculation separates potential and kinetic components. The total energy is

$$E = E_p + E_k,$$

with potential energy relative to the still-water datum

$$E_p = \frac{\rho g}{L} \int_0^L \frac{\eta^2(x)}{2}\, dx,$$

and kinetic energy

$$E_k = \frac{\rho}{L} \int_0^L \int_{-d}^{\eta(x)} \frac{u^2 + v^2}{2}\, dy\, dx.$$

In the GUI report, the main integral quantities are evaluated with the following dimensional expressions:

$$\begin{aligned}
I &= \rho(cd - Q), \\
T_k &= \frac{1}{2}(cI - Q\rho\bar{u}_1), \\
V_p &= \frac{1}{2}\rho g\langle\eta^2\rangle, \\
E &= T_k + V_p, \\
F &= c(3T_k - 2V_p) + \frac{1}{2}\langle u^2_{b,\mathrm{abs}}\rangle(I + \rho cd) - 2c\bar{u}_1 I, \\
S_{xx} &= 4T_k - 3V_p + \rho\langle u^2_{b,\mathrm{abs}}\rangle d - 2\bar{u}_1 I, \\
U_{\mathrm{drift}} &= \frac{I}{\rho d}, \\
C_g &= \frac{F}{E}.
\end{aligned}$$

The integral formulas use the **absolute** bed velocity, including Eulerian current, following Fenton's Nonlinear wave theories. The density factor in kinetic energy is explicit here because

I includes density. The invariant and absolute-bed-velocity expressions coincide for an exact steady wave. At a stored finite Fourier state, the invariant formulas supply the main outputs, and the independently evaluated spectral bed-velocity formulas are compared against them within the budgets in §7.7.

$$\langle u_{b,\mathrm{abs}}^2 \rangle = 2r - c(c - 2\bar{u}_1) = \bar{u}_1^2 + \langle (u_b - \bar{u}_1)^2 \rangle.$$

The orbital mean square reported separately by the GUI is not interchangeable with this absolute mean square in F or S_{xx}. When criterion 2 is selected in a supporting solver interface, U_c is the prescribed Stokes current; the integral formulas still use the solved Eulerian current, $\bar{u}_1$.

The field `Cg` is labelled **energy-flux velocity**, F/E. It is an energy-flux-to-energy-density ratio; it is not a general evaluation of the dispersion derivative $d\omega/dk$ for finite-amplitude waves with current.

For the implemented cosine reconstruction, the last mode has amplitude $Y_N/2$, where Y_j denotes the surface coefficient E_j. Consequently the potential energy of that reconstructed surface is

$$V_p = \frac{\rho g}{k^2} \left(\frac{1}{4} \sum_{j=1}^{N-1} Y_j^2 + \frac{Y_N^2}{16} \right).$$

Acceptance in the Rust/Python/C++ GUI, HTML and VBA solvers requires the residual limits in §7.6 and the resolution and output-accuracy checks in §7.7. A small Newton correction alone is insufficient. The complete target state is checked before accepted engineering outputs are exposed.

The point kernel evaluates the hyperbolic ratios with scaled exponentials to avoid cancellation and overflow at the bed:

$$S(a,b) = \frac{e^a - e^{-a-2b}}{1 + e^{-2b}}, \qquad C(a,b) = \frac{e^a + e^{-a-2b}}{1 + e^{-2b}}, \qquad a = jY,\ b = jkd.$$

These quantities should be interpreted with the same current convention used in the solve. The Eulerian-current and Stokes-current closures lead to different fixed-frame flux and drift interpretations even when the relative wave form is similar.

The orbital mean square bed velocity diagnostic removes the solved Eulerian current:

$$u_{b2} = \left\langle (u_{\mathrm{bed}}(t) - \bar{u}_1)^2 \right\rangle.$$

The stability diagnostics include the Miche limiting-height estimate,

$$H_{\mathrm{Miche}} = 0.142 L \tanh(kd),$$

and the saturation ratio,

$$\text{saturation} = \frac{H}{H_{\mathrm{Miche}}}.$$

The Miche estimate is a practical screening formula rather than the exact limiting wave curve. Near-limiting cases should also be judged by convergence behavior, crest sharpness, Fourier-coefficient decay and the physical plausibility of the branch selected in current.

Output group	Examples	Use
Geometry	L, k, c	Defines the solved wave.
Free surface	η_m, Fourier amplitudes	Crest/trough shape and asymmetry.
Kinematics	u, v, acceleration	Loads, velocity checks and flow fields.
Integrals	I, E, F, S_{xx}	Momentum, energy, power and radiation-stress diagnostics.
Stability	Ur, H_{Miche}, saturation	Practical validity screening.

8.1 Independent pressure and integral checks

Using the mean-level Bernoulli constant r, the dimensional pressure is reconstructed as

$$p = \rho\left[r - gy - \frac{1}{2}\left((u-c)^2 + v^2\right)\right].$$

With $\langle\cdot\rangle$ denoting an average over one wavelength, the independent defining-integral checks include

$$I = \left\langle \int_{-d}^{\eta} \rho u \, dy \right\rangle,$$

$$S_{xx} = \left\langle \int_{-d}^{\eta} \left(p + \rho u^2\right) \, dy \right\rangle - \frac{1}{2}\rho g d^2,$$

$$F = \left\langle \int_{-d}^{\eta} \left[p + \rho g y + \frac{1}{2}\rho(u^2+v^2)\right] u \, dy \right\rangle.$$

The kinetic- and potential-energy integrals are defined above. These checks use fixed-frame absolute velocities, including the solved Eulerian current. `MomentumFlux`, a separate moving-frame momentum-flux diagnostic, is not interchangeable with `Sxx` and is not one of the nine interval-enclosed outputs.

8.2 Phase extrema and engineering estimates

The Rust/Python/C++ GUI and HTML solvers estimate phase maxima by sampling $\max(257, 8N+1)$ points on the symmetric half-wave and refining each detected local-maximum bracket with golden-section search. Refinement uses at most 64 iterations or stops when the phase bracket is narrower than 10^{-12} radians. The largest evaluated magnitude is retained. This provides refined numerical estimates, rather than a proof of the global maximum between all sampled points.

Field	Meaning
`u_bed`	Estimated maximum absolute horizontal velocity at the bed, including current.
`u_surf`	Estimated maximum absolute horizontal velocity evaluated on the reconstructed surface.
`w_max`	Estimated maximum absolute vertical velocity evaluated on that surface.
`acc_max`	Estimated maximum absolute material horizontal acceleration evaluated on that surface.
`u_bed_crest`, `u_surf_crest`	Signed horizontal velocities at crest phase; distinct from magnitude maxima.
`extrema_phases`	Phases at which the retained maxima estimates were evaluated.

The material horizontal acceleration used by the GUI and browser is

$$a_x = \frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = (u - c)\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y}.$$

This differs from the local acceleration $\partial u/\partial t = -c\,\partial u/\partial x$ written by the standalone flow-field drivers in §11.7.

The engineering bed diagnostics are

$$\tau_{\mathrm{bed,est}} = \frac{1}{2}\rho C_f\, u_{\mathrm{bed}}^2, \qquad C_f = 0.005,$$

$$A_{\mathrm{bed,est}} = \frac{T}{2\pi} U_{b,\mathrm{orb,est}}.$$

Here $U_{b,\mathrm{orb,est}}$ is the estimated phase maximum of $|u_b(X) - \bar{u}_1|$. `ExcursionBed` therefore uses the orbital velocity after removing the Eulerian current. It is an amplitude-based excursion estimate, not direct integration of the nonlinear particle trajectory. Bed shear likewise remains an estimate using a fixed friction coefficient. Miche screening, bed shear and phase extrema are engineering diagnostics outside the nine selected-output interval checks.

---

9. Wavelength-only kernels and surrogate formulas

The repository includes approximate layers for speed, spreadsheet deployment and design charts. These formulas are placed here because they are computational derivatives of the same physical mapping $L = F(H, T, d, U_c)$.

9.1 Current-modified linear baseline

For wavelength-only calculations, a common baseline solves

$$(H, T, d, U_c) \mapsto L.$$

With a collinear current, the baseline residual in k is

$$F(k) = \left(\frac{2\pi}{T} - kU_c\right)^2 - gk\tanh(kd).$$

Newton iteration is then

$$k_{n+1} = k_n - \frac{F(k_n)}{F'(k_n)},$$

where

$$F'(k) = -2U_c\left(\frac{2\pi}{T} - kU_c\right) - g\tanh(kd) - gkd\frac{1}{\cosh^2(kd)}.$$

The wavelength is recovered from

$$L = \frac{2\pi}{k}.$$

Without current, the same baseline can be expressed as the fixed-point wavelength equation

$$L_{\text{lin}} = \frac{gT^2}{2\pi}\tanh\left(\frac{2\pi d}{L_{\text{lin}}}\right).$$

9.2 Common surrogate structure

Most reduced-order models correct a physically meaningful baseline:

$$L = L_{\text{base}}C(\text{x}).$$

Typical nondimensional features include

$$\frac{H}{L}, \qquad \frac{d}{L}, \qquad kd, \qquad \frac{U_c}{c}, \qquad \frac{U_c}{\sqrt{gd}}.$$

The implemented engineering feature set includes

$$
\begin{aligned}
\text{WaveSteepness} &= \frac{H}{L_{\text{lin}}},\\
\text{RelativeDepth} &= \frac{d}{L_{\text{lin}}},\\
\text{DopplerFactor} &= \frac{U_c T}{L_{\text{lin}}},\\
\text{UrsellNumber} &= \frac{H L_{\text{lin}}^2}{d^3},\\
\text{CurrentFroude} &= \frac{U_c}{\sqrt{gd}}.
\end{aligned}
$$

For symbolic-regression and neural workflows, additional logarithmic and current features may be used:

$$
\begin{gathered}
x_0 = \ln\!\left(\frac{d}{L}\right), \qquad x_1 = \ln\!\left(\frac{H}{L}\right), \qquad x_2 = \ln\!\left(\frac{H}{d}\right),\\
x_3 = \ln(Ur), \qquad Ur = \frac{HL^2}{d^3},\\
x_4 = \frac{U_c}{\sqrt{gd}}, \qquad x_5 = \frac{U_c T}{L}, \qquad x_6 = \frac{U_c}{C_0},
\end{gathered}
$$

with the deep-water scale quantities

$$
C_0 = \frac{gT}{2\pi}, \qquad L_0 = \frac{gT^2}{2\pi},
$$

and the additional features

$$
x_7 = \ln\!\left(\frac{H}{L_0}\right), \qquad x_8 = \ln\!\left(T\sqrt{\frac{g}{d}}\right).
$$

9.3 Pade/rational surrogate

The Pade surrogate represents a correction with a rational function,

$$
P(x) = \frac{a_0 + a_1 x + a_2 x^2 + \cdots + a_m x^m}{1 + b_1 x + b_2 x^2 + \cdots + b_n x^n}.
$$

The depth-regime variable is usually

$$
\mu = \frac{d}{L}.
$$

The regime split used for training and interpretation is

$$
\begin{aligned}
\mu < 0.05 \quad &\text{(shallow)},\\
0.05 \le \mu < 0.5 \quad &\text{(intermediate)},\\
\mu \ge 0.5 \quad &\text{(deep)}.
\end{aligned}
$$

The resulting Pade wavelength expression is

$$L = L_{\text{base}} C_{\text{Pade}}(\mathrm{x}),$$

or explicitly

$$L = L_{\text{base}} \frac{p_0 + p_1 x_1 + p_2 x_2 + \cdots}{1 + q_1 x_1 + q_2 x_2 + \cdots}.$$

9.4 Neural-network surrogate

The neural surrogate is a global tanh-network approximation,

$$\hat{y}(\mathrm{x}) = b_o + \sum_{j=1}^{M} w_j^{(o)} \tanh\left(b_j + \sum_{i=1}^{p} w_{ji}^{(h)} x_i \right).$$

The inputs may be standardized as

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i},$$

or min-max scaled as

$$\tilde{x}_i = \frac{x_i - x_i^{\min}}{x_i^{\max} - x_i^{\min}}.$$

Training minimizes mean-square loss,

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} (y_n - \hat{y}_n)^2,$$

and prediction quality is commonly summarized by

$$\text{MAPE} = \frac{100}{N} \sum_{n=1}^{N} \left| \frac{y_n - \hat{y}_n}{y_n} \right|.$$

9.5 Genetic symbolic-regression formula

The genetic symbolic-regression model is an explicit correction over a baseline,

$$L \approx L_{\text{base}} C_{\text{GEP}}(\mathrm{x}),$$

or, more generally,

$$L \approx G(\mathrm{x}).$$

The documented explicit expression is

$$L_{\text{genetic}} = L_{\text{lin}} \left[\exp\left(\frac{345 U_c T}{509 L_{\text{lin}}} \right) + \frac{1052\pi U_c}{1052\pi U_c + 435 g T} - \frac{5 \tanh\left(\frac{U_c T}{L_{\text{lin}}} \ln\left(T\sqrt{\frac{g}{d}} \right) + \frac{2117}{961} \right)}{67 \ln\left(\frac{H}{d} \right)} \right].$$

For readability and implementation checking, it can be decomposed as

$$L_{\text{genetic}} = L_{\text{lin}} M,$$

where

$$\begin{gathered} M = t_1 + t_2 + t_3, \\ t_1 = \exp\left(\frac{345 U_c T}{509 L_{\text{lin}}} \right), \qquad t_2 = \frac{1052\pi U_c}{1052\pi U_c + 435 g T}, \\ t_3 = -\frac{5 \tanh\left(\frac{U_c T}{L_{\text{lin}}} \ln\left(T\sqrt{\frac{g}{d}} \right) + \frac{2117}{961} \right)}{67 \ln\left(\frac{H}{d} \right)}. \end{gathered}$$

9.6 Effective transformed-variable formula

The effective-variable model transforms the input period and depth, then solves a linear dispersion relation with transformed variables. The transformed period and depth are

$$\begin{aligned} T_{\text{eff}} &= T - \frac{93}{943} \left(U_c^2 - \frac{H}{1819 \cdot 708} \right) + \frac{500}{739} U_c, \\ d_{\text{eff}} &= d + \frac{H^{3/4}}{d^{1/4}} + \frac{986}{873} U_c \tanh(H). \end{aligned}$$

Equivalently,

$$T_{\text{eff}} = T + \Delta_T, \qquad d_{\text{eff}} = d + \Delta_d,$$

with

$$\begin{aligned} \Delta_T &= \left(U_c^2 - \frac{H}{1819 \cdot 708} \right) \left(-\frac{93}{943} \right) + \frac{500}{739} U_c, \\ \Delta_d &= \sqrt{\frac{H}{\sqrt{d/H}}} + \frac{986}{873} U_c \tanh(H) = \frac{H^{3/4}}{d^{1/4}} + \frac{986}{873} U_c \tanh(H). \end{aligned}$$

The algebraic identity in the depth transformation is

$$\sqrt{\frac{H}{\sqrt{d/H}}} = \frac{H^{3/4}}{d^{1/4}}.$$

The final effective wavelength is obtained from

$$L_{\text{effective}} = \mathcal{L}_{\text{lin}}(T_{\text{eff}}, d_{\text{eff}}),$$

where

$$\left(\frac{2\pi}{T_{\text{eff}}}\right)^2 = gk\tanh(kd_{\text{eff}}), \qquad L_{\text{effective}} = \frac{2\pi}{k}.$$

The transformed period must remain nonzero:

$$T_{\text{eff}} \neq 0.$$

---

10. Program features

Feature	Description
Primary native Rust application	**`fenton_gui.rs`: Rust 2021; native Windows GNU x64 GUI, independent CLI build, verification certificates, direct `rustc` compilation and no external crates.**
Secondary Python application	**`fenton_gui.py`: Tkinter + NumPy, optional Numba acceleration, direct Python execution and PyInstaller GUI executable.**
Fully nonlinear stream-function solve	Solves surface shape, velocity field, wavelength and integral quantities.
Current-aware formulation	Separates Eulerian, Stokes, mean-flow and flux conventions.
Continuation	Adapts height increments and restores the last accepted state after a failed trial.
Fourier order	Requested $N = 50$ in the precision driver; the accepted order is limited by Jacobian conditioning and reported explicitly.
Numerical verification	Rust, Python/C++ GUIs, HTML and VBA: finite-state interval enclosures, surface-resolution checks and independent output comparisons. Standalone drivers: their documented residual and spectral diagnostics.
Lean evidence	Mathematical identities, finite-system implications and recorded inequalities; scope and benchmark coverage in §7.8.
Short-period calculations	Stable vertical basis and adaptive continuation/order control support tested 3 s and 4 s cases without guaranteeing every height/current combination.

Feature	Description
GUI output	Rust and Python report accepted geometry, estimated phase extrema, integrals, accuracy diagnostics and breaking-screening quantities; Rust uses a fixed-width 107-character text report.
Wavelength API	Provides callable `L(H,T,d,Uc)` for scripts and tables.
Surrogates	Provides Pade, neural, symbolic-regression and effective-variable approximations.
Excel/VBA support	`L_fenton` returns an accepted nonlinear wavelength; `Fenton_report` exposes checks and enclosures. Separate modules provide their respective approximations.
Android availability	Fenton App is also available as the installable package `fenton.apk`.
Nomograms	Generates design charts for rapid engineering use.

---

11. Installation and use

11.1 Main application — Rust GUI and command-line solver

11.1.1 Rust implementation and platform requirements The main program is `fenton_gui.rs`, a single-file **Rust 2021 edition** application. It contains the complete wave solver, report generator, numerical tests and Win32 GUI. Its standard build produces a native Windows x64 executable named `fenton_gui.exe`; a conditional `headless` build produces a command-line program from the **same source file**. No Python, NumPy, Numba, Cargo dependencies, Visual Studio, Microsoft C++ Build Tools, MSVC linker or WinGet is needed for the Rust compilation.

Requirement	Production configuration
Build operating system	Windows 10 64-bit (version 1809 or later) or Windows 11 64-bit
Programming language	Rust, **2021 edition** (`--edition=2021`)
Compiler distribution	Rust `rustc` supplied by the **MSYS2 UCRT64** package repository
Toolchain target / host	`x86_64-pc-windows-gnu`
Linker	GNU GCC driver from the UCRT64 environment (`-C linker=gcc`)
Rust crates	Rust standard library only; no third-party crates and no mandatory `Cargo.toml`
Graphical user interface	Native Win32 (`user32`, `gdi32`, `kernel32`), with no external GUI framework
Floating-point precision	IEEE-754 binary64 (`f64`); no fast-math or relaxed reassociation

Requirement	Production configuration
Release build	`-C opt-level=3`, `-C codegen-units=1`, `-C lto=fat`, `-C panic=unwind`
Execution outputs	GUI display and UTF-8 `output.txt` in the current working directory

The Rust compiler **edition is not a fixed compiler-version pin**. MSYS2's `pacman` installs the Rust release currently available in the configured repository. Record the exact installed build with `rustc --version` and `rustc -vV` when a reproducible engineering calculation is needed. The project is compiled directly by `rustc`; a `Cargo.toml` and a separate `rustup` installation are not required for this build method.

The current MSYS2 installer and UCRT64 build environment target modern Windows. These commands **are not a Windows 7 installation procedure**. A Rust release compiled on one processor must also respect the selected CPU target when copied to another computer.

11.1.2 Install MSYS2 and the complete UCRT64 toolchain

1. Download the x64 MSYS2 installer from msys2.org and install it, normally to `C:\msys64`.
2. Launch **MSYS2 UCRT64** from the Windows Start menu, not the plain MSYS2/MSYS terminal and not an unrelated MINGW64 shell.
3. Update the package database and installed packages:

```
pacman -Syu
```

If requested, close the terminal, reopen **MSYS2 UCRT64**, and repeat `pacman -Syu` until the update finishes normally.

4. Install the GNU compiler, binutils, Rust compiler and Cargo in the same UCRT64 environment:

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
↪ mingw-w64-ucrt-x86_64-rust
```

The `mingw-w64-ucrt-x86_64-rust` package is the Windows GNU/UCRT64 Rust distribution. In particular, `pacman -S rust` **alone is not the same installation**: that selects the MSYS runtime package rather than the native UCRT64 Rust package. Keep the Rust compiler, GCC linker, C runtime and libraries within the same `/ucrt64` prefix; do not mix `/mingw64` and `/ucrt64` binaries or add an unrelated `%USERPROFILE%\.cargo\bin` to the front of the UCRT64 `PATH`.

Check the environment before compiling:

```
echo "$MSYSTEM"
command -v rustc
command -v cargo
command -v gcc
command -v objdump
rustc --version
cargo --version
gcc --version
rustc -vV
rustc --print target-libdir --target x86_64-pc-windows-gnu
```

Expected values include `UCRT64`, paths under `/ucrt64/bin/`, and `host: x86_64-pc-windows-gnu`. If these differ, resolve the toolchain-selection issue before compiling. Compiler updates may change floating-point last digits and should be followed by reference tests.

11.1.3 Project directory and direct GUI compilation Put the Rust source in the intended build directory using the exact filename `fenton_gui.rs`. For example, a project located at `C:\Downloads\github\fenton-nolinear-calculator-main\rust` is accessed inside MSYS2 as:

```
cd /c/Downloads/github/fenton-nolinear-calculator-main/rust
ls -lh fenton_gui.rs
```

The **recommended machine-optimized release command**, to be run in **MSYS2 UCRT64 Bash**, is:

```
rustc --edition=2021 --target x86_64-pc-windows-gnu \
  -C linker=gcc \
  -C opt-level=3 \
  -C codegen-units=1 \
  -C lto=fat \
  -C target-cpu=native \
  -C target-feature=-fma \
  -C prefer-dynamic=no \
  -C panic=unwind \
  -C strip=symbols \
  "fenton_gui.rs" -o "fenton_gui.exe"
```

Copy/paste the **complete command**, including the final source and `-o` arguments. In Bash, each continuation backslash must be the last character on its line. To avoid multi-line quoting errors, the equivalent one-line command is:

```
rustc --edition=2021 --target x86_64-pc-windows-gnu -C linker=gcc -C opt-level=3 -C
 ↪  codegen-units=1 -C lto=fat -C target-cpu=native -C target-feature=-fma -C
 ↪  prefer-dynamic=no -C panic=unwind -C strip=symbols "fenton_gui.rs" -o
 ↪  "fenton_gui.exe"
```

The `target-cpu=native` executable is tuned for the build machine's CPU and **must not be assumed executable on older x64 processors**. For a general-purpose x64 build use the following portable baseline:

```
rustc --edition=2021 --target x86_64-pc-windows-gnu -C linker=gcc -C opt-level=3 -C
 ↪  codegen-units=1 -C lto=fat -C target-cpu=x86-64 -C prefer-dynamic=no -C
 ↪  panic=unwind -C strip=symbols "fenton_gui.rs" -o "fenton_gui_portable.exe"
```

The portable binary uses baseline x86-64 instruction selection but still requires a compatible Windows OS and any runtime DLLs shown by the executable's import table. The source's `windows_subsystem = "windows"` conditional attribute makes this a normal GUI executable without a separate console window. No `-mwindows` or `WinMain` wrapper is needed.

Check and run the binary:

```
test -f fenton_gui.exe && echo "GUI executable built"
./fenton_gui.exe
```

Enter wave height H [m], period T [s], still-water depth d [m] and signed Eulerian current U_c [m/s]; press **CALCULATE**. The initial values are $H = 3$ m, $T = 9$ s, $d = 5$ m and $U_c = 1$ m/s. Positive current follows the wave-propagation direction. For nonzero current, the program

evaluates the no-current and imposed-current cases separately. The 107-character-wide report is displayed in the native GUI and saved to `output.txt` in the process's working directory. The file uses UTF-8; the Win32 text viewer displays UTF-16 with Windows line endings. A write failure can prevent file creation even when the GUI displays a completed report.

11.1.4 Compiler flag reference and floating-point safety

Option	Meaning and engineering consequence
`--edition=2021`	Parse the Rust 2021 source correctly; independent of the installed `rustc` release number.
`--target x86_64-pc-windows-gnu`	Generate a 64-bit native Windows executable using the GNU ABI.
`-C linker=gcc`	Use the GCC linker driver from the selected MSYS2 UCRT64 toolchain.
`-C opt-level=3`	Enable high LLVM optimization without requesting unsafe fast-math semantics.
`-C codegen-units=1`	Build one code-generation unit to enable wider optimization; compilation can take longer.
`-C lto=fat`	Whole-program link-time optimization across the application and eligible Rust library code.
`-C target-cpu=native`	Tune machine instructions and scheduling for the local CPU; not necessarily portable and not guaranteed faster for every case.
`-C target-cpu=x86-64`	Produce the baseline x86-64 alternative intended for broader CPU compatibility.
`-C target-feature=-fma`	Avoid FMA instruction contraction in the optimized native build, preserving the implementation's separate multiply/add rounding model used by its binary64 interval checks.
`-C prefer-dynamic=no`	Prefer static Rust libraries where supported; does not itself statically link all GCC/MinGW libraries.
`-C panic=unwind`	Preserve unwinding needed by `std::panic::catch_unwind` around numerical verification and GUI worker logic.
`-C strip=symbols`	Remove debugging symbols from the release executable; numerical results are not intentionally changed.
`-o fenton_gui.exe`	Write the compiled GUI program to the specified filename.

Do not enable fast-math, finite-math-only assumptions, floating-point reassociation, or `panic=abort`. In particular, `panic=abort` would bypass the source's intended `catch_unwind` handling. The interval checker expects IEEE binary64 with gradual underflow; the source

performs a runtime subnormal check. High compiler optimization is compatible with the target verification method only while those arithmetic assumptions remain valid. A new compiler/CPU configuration should be evaluated using the built-in tests and representative nonlinear-wave cases before engineering use.

11.1.5 Compile and run the optional Rust command-line solver The same source implements a console entry point behind `feature="headless"`. Compile it in MSYS2 UCRT64 without the GUI subsystem:

```
rustc --edition=2021 --target x86_64-pc-windows-gnu \
  --cfg 'feature="headless"' \
  -C linker=gcc -C opt-level=3 -C codegen-units=1 -C lto=fat \
  -C target-cpu=x86-64 -C prefer-dynamic=no -C panic=unwind \
  -C strip=symbols "fenton_gui.rs" -o "fenton_cli.exe"
```

Use four positional numbers in the order `H T d Uc`, all in SI units. For example:

```
./fenton_cli.exe 3.0 9.0 5.0 1.0
```

When no arguments are supplied, the code uses the default wave `(3.0, 9.0, 5.0, 1.0)`. It prints the engineering report to standard output and writes `output.txt` in the current working directory. The CLI build is useful for batch scripts, independent checks and timing comparisons. Do not assume that positional arguments will be read by the GUI executable; the GUI reads its input controls.

11.1.6 Unit tests, numerical reference cases and release validation Build the test harness directly from the same Rust source:

```
rustc --edition=2021 --test --target x86_64-pc-windows-gnu \
  -C linker=gcc -C panic=unwind "fenton_gui.rs" -o "fenton_tests.exe"
./fenton_tests.exe
./fenton_tests.exe --ignored
```

The ordinary test suite includes invalid-input rejection, Jacobi SVD consistency, initialization and fixed-width report formatting. The **ignored** tests exercise full nonlinear calculations and reference wavelengths/power, and therefore require explicit `--ignored`. In the supplied Rust source, reference cases cover the no-current case and positive/negative-current configurations, including a 50-mode $H = 3$ m, $T = 9$ s, $d = 5$ m, $U_c = 1$ m/s case. Numerical reference tolerances apply to the computed quantities; formatted report values are rounded to five decimals for display only.

For a complete release check, build the `native` and `portable` variants, run the tests with each compiler/toolchain configuration, inspect `output.txt` for the default case and compare **acceptance status, active Fourier order, residuals and all engineering outputs** within the implementation's stated numerical budgets. File-by-file byte equality is not required when different floating-point math libraries, CPU instructions or summation sequences produce harmless last-bit differences. Numerical acceptance decisions and output tolerances, rather than text equality alone, govern engineering equivalence.

If a `build_msys2_ucrt64.sh` script is included in the local distribution, its `gui`, `portable`, `all` and `bench` modes can automate the corresponding build/check operations; the **direct commands above do not depend on that optional script**.

11.1.7 Native DLL dependencies and deployment A Rust executable built for Windows GNU normally depends on native Windows DLLs such as `KERNEL32.dll`, `USER32.dll`, `GDI32.dll` and the system C runtime. Examine imports before distributing a compiled executable:

```
objdump -p fenton_gui.exe | grep -i 'DLL Name'
```

`-C prefer-dynamic=no` does **not** prove that all GNU runtime or threading DLLs are statically linked. If imports include `libwinpthread-1.dll`, `libgcc_s_seh-1.dll` or another non-system DLL, deploy a compatible permitted redistribution or investigate a static native-library build, and then retest the resulting binary. A static-linking attempt with GCC support libraries is toolchain-dependent:

```
rustc --edition=2021 --target x86_64-pc-windows-gnu -C linker=gcc -C opt-level=3 -C
↪ codegen-units=1 -C lto=fat -C target-cpu=x86-64 -C prefer-dynamic=no -C
↪ panic=unwind -C strip=symbols -C link-arg=-static -C link-arg=-static-libgcc
↪ "fenton_gui.rs" -o "fenton_gui_static.exe"
```

Static GNU linkage cannot remove ordinary Windows system DLL imports, and the optional flags may be rejected by some package combinations. An executable is standalone only with respect to the **external libraries it actually imports**; check the PE import table and test on a clean target machine. Building the application does not install software on the end-user computer, and the Rust compiler and MSYS2 themselves are not needed there if all non-system binary dependencies have been provided or eliminated.

11.1.8 Compiling through PowerShell and troubleshooting MSYS2 UCRT64 Bash is the recommended compilation terminal. From an existing Windows PowerShell session, a properly configured MSYS2 installation can be entered with:

```
$env:MSYSTEM = 'UCRT64'
$env:CHERE_INVOKING = 'yes'
& 'C:\msys64\usr\bin\bash.exe' -l
```

Inside that Bash session, run the installation checks and `rustc` compilation commands from §11.1.3. If an optional local `build_msys2_ucrt64.sh` script exists, it can be called through `bash -lc` after setting those variables; a script is not required for the primary direct compilation workflow.

Symptom	Recommended action
`rustc: command not found`	Verify `echo "$MSYSTEM"` gives `UCRT64`; run `pacman -S --needed mingw-w64-ucrt-x86_64-rust` and recheck the executable path.
`gcc: command not found` or linker failure	Install the `mingw-w64-ucrt-x86_64-toolchain` package group; check `command -v gcc` resolves under `/ucrt64/bin`.
`can't find crate for std`	Confirm `rustc -vV`, target triple and `rustc --print target-libdir`; eliminate mixed or incomplete Rust installations.

Symptom	Recommended action
Unexpected C runtime or missing GNU DLL	Inspect imports with `objdump -p`; ensure all compiler/linker tools belong to the same UCRT64 prefix and deployment dependencies are present.
PowerShell syntax error on \	Use the Bash terminal for commands containing \ line continuations; PowerShell uses a different continuation character.
Report is visible but `output.txt` is absent	Check write permissions and the process working directory; the report is written using a relative filename.
`panic=abort` or failed interval verification	Recompile with `-C panic=unwind`, keep `f64` and non-fast-math arithmetic, and run the test harness.
Unexpected numerical difference between machines	Compare toolchain versions, CPU targets, runtime libraries, current convention, Fourier order and numerical acceptance diagnostics.

Official toolchain documentation: MSYS2 installation, MSYS2 environments, MSYS2 Windows support, MSYS2 UCRT64 Rust package, and Rust compiler code-generation options.

11.2 Secondary application — Python GUI

```
python -m venv build_env
build_env\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -U "numpy>=1.20"
python -m pip install -U "numba>=0.57"
python fenton_gui.py
```

Numba is optional. If Numba is unavailable, the solver falls back to the pure-NumPy path. The interval checker uses the standard-library `decimal` module. Tk bindings are required by the GUI; the independent verification package additionally requires its listed audit dependencies.

11.3 Build the secondary Python standalone executable

```
pip install pyinstaller
python -m PyInstaller --noconfirm --clean --onefile --windowed --name fenton_gui.exe
↪  fenton_gui.py
```

The executable is written to the `dist` directory. `--onefile` bundles the Python runtime and packaged dependencies; `--windowed` suppresses the console window on Windows. The PyInstaller build requires Python, Tkinter, NumPy and the applicable installed Python packages **on the build machine**. A second explicit naming form is:

```
python -m PyInstaller --noconfirm --clean --onefile --windowed --name fenton_gui
↪  fenton_gui.py
```

This creates a Windows executable named `fenton_gui.exe` inside `dist` in the standard PyInstaller naming convention. If distributing both Rust and Python implementations, place the

Python executable in a separately named folder to avoid overwriting the Rust `fenton_gui.exe`. Python `--self-test` remains available before packaging.

11.4 Additional implementation — build the C++ GUI executable

```
g++ fenton_gui.cpp -o fenton_gui.exe -O3 -std=c++20 ^
   -fno-fast-math -ffp-contract=off ^
   -march=native -flto=auto -fopenmp -static -static-libgcc ^
   -static-libstdc++ -mwindows -pthread ^
   -lgdi32 -luser32 -lkernel32 -lcomctl32
```

Use a Windows toolchain providing the listed libraries. The interval checker requires IEEE binary64 round-to-nearest arithmetic: do not enable `-ffast-math`, `-Ofast` or floating-point reassociation. The source rejects fast-math builds and checks the arithmetic environment at runtime. Cross-implementation comparisons use explicit tolerances rather than requiring identical last digits.

11.5 Wavelength-only use

Use `function.py` when only the wavelength is required by another program or table. The intended callable form is:

```
L(H, T, d, Uc)
```

11.6 Fenton App for Android

The **Fenton App for Android** is distributed as the universal release package `fenton.apk`. It supports Android 6.0 (API 23) and newer and includes ARM64, ARM32, x86 and x86-64 native libraries.

Install `fenton.apk`

1. **Obtain the complete APK.** Download `fenton.apk` directly to the phone, or copy it from a computer into the phone's **Download** folder. Make sure the filename still ends in `.apk`; some messaging and cloud-storage apps rename or incompletely transfer executable files.
2. **Authorize the app that will open the APK.** Android grants local-install permission separately to each source application. Search Android Settings for **Install unknown apps**, **Install unknown applications** or **Installation sources**, select the browser or file manager that will open `fenton.apk`, and enable **Allow from this source**.
3. **Use that same authorized application.** If permission was granted to the file manager, open the phone's file manager, navigate to **Download**, and tap `fenton.apk`. If permission was granted to the browser, open the completed download from that browser. Authorizing a browser does not authorize a file manager, and vice versa.
4. **Approve the Android installer.** Tap **Install** and allow any device security scan to finish. When Android reports that installation is complete, tap **Open** or launch **Fenton Wave Solver** from the app list.
5. **Optionally revoke the source permission.** After installation, **Allow from this source** can be disabled again without affecting the installed application.

Troubleshooting installation

Message or symptom	Recommended action
Installation canceled, **Not allowed** or the installer immediately closes	Confirm that **Allow from this source** is enabled for the exact app currently opening the APK. File Manager, Files, Downloads, Chrome and other browsers each have separate permission settings. Return to the APK and try again after enabling it.
The APK is visible but does not open	Move it to the local **Download** folder and open it with the system file manager. Confirm that its name is `fenton.apk`, not `fenton.apk.zip`, and that the transfer completed.
App not installed while an older Fenton build is present	The installed copy may use another signing certificate or may be newer than the APK. Install a newer signed release, or uninstall the old copy before installing. Uninstalling removes that app's locally stored data.
Invalid package, **Problem parsing the package** or a zero-byte/partial file	Delete the phone copy, download or transfer it again, and compare its size or SHA-256 checksum with the value published for that release.
Installation is blocked on a work, school or supervised phone	A device-administrator policy may prohibit sideloading. Only the administrator can change that policy.
Installation still fails	Check that the device runs Android 6.0 or newer, has adequate free storage, restart it, and retry from the system file manager.

Menu names vary between Android versions and manufacturers. A common path is **Settings > Apps > Special app access > Install unknown apps**, but using Settings search is usually faster.

For advanced diagnosis with Android Platform Tools and USB debugging enabled, run:

```
adb install -r fenton.apk
```

This command prints the precise Android package-manager error when a graphical installer shows only a generic failure message. It is not required for normal installation.

11.7 Standalone precision solver

Compile the `fourier.cpp` source with a C++20 compiler:

```
g++ -std=c++20 fourier.cpp -O3 -march=native -flto=auto -fopenmp -static-libgcc
 ↪  -static-libstdc++ -o fourier.exe
```

Add `-fopenmp` for an OpenMP build where supported. On Windows, use `-o fourier.exe`. The source uses the same driver and result-file contract; substitute that source filename when compiling it.

The program reads `data.dat` from the working directory or, if absent there, beside the executable. If no file is found, it requests all required case values interactively. Physical inputs have no silent defaults.

`data.dat` contains exactly seven non-empty lines:

Line	Contents
1	Case title
2	Relative wave height $H/d > 0$
3	`Period` or `Wavelength`
4	$T\sqrt{g/d} > 0$ for `Period`, or $L/d > 0$ for `Wavelength`
5	Current criterion: `1` for Eulerian, `2` for mass transport
6	Signed prescribed current divided by $\sqrt{gd}$
7	`FINISH`

The requested order is 50; the driver selects its initial continuation subdivision internally. Do not add Fourier-order or height-step lines to this file format.

For $H = 3$ m, $T = 9$ s, $d = 5$ m, zero Eulerian current and $g = 9.80665$ m/s^2, use:

```
H=3 m, T=9 s, d=5 m, no current
0.6
Period
12.604274275022739
1
0.0
FINISH
```

Run:

```
./fourier.exe
```

On Windows, run `fourier.exe`. `fourier --help` displays the input contract. When using `data.dat`, results are written beside that file; interactive runs use the working directory.

File	Contents
`solution.res`	Wave geometry, currents, integral quantities and native Fourier coefficients.
`solver.res`	Order selection, conditioning, continuation history, complete state/residual vectors and convergence/resolution checks.
`surface.res`	Surface samples: X_{dim}/d, bed-based elevation y/d, and pressure divided by $\rho g d$.
`flowfield.res`	Bed-to-surface profiles of velocity, local acceleration, velocity gradients and Bernoulli diagnostics.

The `eta/d` label in `surface.res` denotes the **bed-based** ordinate $y/d = 1+\eta/d$: the bed is at 0 and the mean water level at 1. Local accelerations in `flowfield.res` are $\partial u/\partial t$ and $\partial v/\partial t$, not

material accelerations including convective terms. The orbital bed-velocity statistic excludes the solved Eulerian current.

Inspect the process exit status and `solver.res` before using results. A nonzero exit status means the requested run did not complete successfully; old `.res` files are not evidence of a successful new calculation.

11.8 Standalone SVD solver

`fourier_carvalho.cpp` retains a separate command-line and file interface:

```
g++ -std=c++20 -O3 -Wall -Wextra -pedantic fourier.cpp -o fourier.exe
./fourier.exe 0.6 Period 12.604274275022739 1 0.0 50 4
```

The arguments are `H_over_d Case Value CurrentCriterion [CurrentValue] N Steps`. `CurrentValue`, when present, follows the current criterion. `N` is the requested Fourier order, from 1 to 512; `Steps` is the initial height subdivision, from 1 to 4096, and continuation subsequently adapts its increments. If `CurrentValue` is omitted, the current is zero.

A discovered `data.dat` takes precedence over positional arguments. This driver reads the title and the same five physical fields as above, followed by **two additional lines**, `N` and `Steps`; a trailing `FINISH` may follow. Use the format for the executable being run: this input is not interchangeable with the precision driver's seven-line file. The SVD driver also retains its separate deep-water branch.

Successful runs write `solution.res`, `surface.res` and `flowfield.res`. Numerical diagnostics are included in `solution.res`; this driver does not produce `solver.res`.

11.9 Browser interface

Open the self-contained `fenton.html` file in a browser and enter H, T, d and Eulerian current U_c in SI units. Calculation runs in a Web Worker. The interface shows accepted engineering values, the active Fourier order, residual bounds and output-accuracy diagnostics. The no-current and imposed-current cases are accepted separately; a rejected column is suppressed without hiding an accepted comparison case.

Use **Full report** for the numerical method, diagnostic ratios and selected-output enclosures. **Download TXT** exports the readable report; **Download JSON** exports the input, solver statuses, Fourier state and verification data. JSON may contain diagnostic states for rejected cases; use the acceptance flags before treating any values as engineering results. **Clear output** cancels a pending calculation and clears the displayed results.

JavaScript uses binary64 arithmetic, so final machine-level digits can vary between engines. The recorded numerical comparison in §7.8 executes the solver under Node.js; it does not assert identical execution in every browser.

11.10 Excel/VBA nonlinear wavelength and verification report

Import `fenton.bas` into a **standard VBA module** through the Visual Basic Editor's **File > Import File** command, and save the workbook as `.xlsm`. The module uses SI inputs, $g = 9.80665$ m/s² and, for the internal integral checks, $\rho = 1025$ kg/m³. Its prescribed current is Eulerian.

The wavelength function retains this interface:

```
L_fenton(H, T, d, [U])
```

`U` defaults to zero. Adaptive height continuation is always applied. A successful call returns wavelength in metres. A failed nonlinear solve, inadmissible state or failed accuracy check returns an `Err:` text message, rather than a partial or unverified wavelength.

```
=L_fenton(3,9,5)
=L_fenton(3,9,5,1)
```

Use semicolons instead of commas if required by Excel's regional settings.

For the default wave $H = 3$ m, $T = 9$ s and $d = 5$ m, calculated wavelengths are:

Eulerian current (m/s)	Wavelength (m), approximately
0	68.706761529
1	78.827222116

The VBA numerical comparison uses a mechanical C++ translation of the module, with independent high-precision enclosure checks. Native Excel compilation and execution have not been run in that verification environment.

11.11 Python and Lean verification

Run the Python solver's built-in acceptance/rejection checks with:

```
python fenton_gui.py --self-test
```

The companion verification package contains its own source snapshots, dependency requirements, manifests, numerical audit and Lean build configuration. From that package directory, its reproduction sequence is:

```
python -m pip install -r requirements.txt
python audit.py
python check_intervals.py
python check_report.py
python inputs/fenton_gui.py --self-test
python build_certificates.py
python build_finite_certificates.py
lake update
lake exe cache get
lake build
```

The package uses Lean 4.34.0 and Mathlib. The audit checks the source identities before generating recorded certificates. Successful reproduction requires both the numerical tests and all five Lean modules to pass. Deliberate rejection of an under-resolved case is a passing acceptance test when that rejection is the expected outcome.

---

12. Validity and numerical cautions

Condition	Practical implication
Non-breaking steady waves	The formulation is for a periodic wave of permanent form; breaking, strong turbulence and air entrainment are outside the model.
Irrotational flow	Strong shear, vorticity, boundary-layer dominance or rapidly varying currents require a different model.
Horizontal-bed assumption	The steady solver is a local horizontal-bed theory; shoaling over variable bathymetry requires an additional propagation/shoaling model or a local approximation.
Very long waves	Fourier coefficients become broad-banded near the solitary-wave limit. Inspect coefficient decay and between-node defects; a larger requested N is useful only while the system remains well-conditioned.
Near highest waves	Sharp crests degrade convergence and conditioning; inspect the achieved height, active Fourier order, residuals and free-surface shape.
Short periods	For fixed height and depth, a shorter period can produce a much steeper wave. Automatic order reduction addresses conditioning, but cannot establish the existence of a steady solution.
Resolution and acceptance	Small collocation residuals alone do not establish between-node accuracy. GUI/HTML/VBA results are rejected when their accuracy budgets fail. Standalone spectral `FAIL` and nonlinear `PASS` remain separate diagnostics under their documented contracts.
Finite-system verification	Enclosures concern equation and output evaluation at the stored coefficients; they do not include physical-input uncertainty, an exact-root error bound or continuous Fourier-truncation error.
Estimated extrema	Refined phase searches estimate maxima. Bed shear and excursion are engineering estimates with the definitions in §8.2.
Comparing implementations	Match height, depth, period or wavelength, current definition, current value, gravity, active order and tolerances. Differences in quadrature and floating-point linear algebra can prevent exact numerical equality.

Condition	Practical implication
Opposing current	Wave blocking, multiple branches or no physical branch may occur; the chosen root must be checked.
Current criterion	Do not mix Eulerian-current and Stokes/mass-transport current definitions.
Surrogate use	Do not extrapolate outside the training/calibration range. Surrogates are wavelength-estimation tools, not replacements for the nonlinear kinematic solution.
Units	GUI inputs use coherent SI units; standalone `data.dat` inputs are the dimensionless groups specified in §11.7.

Fenton's theory comparisons give the following practical hierarchy. Fifth-order Stokes theory is useful for shorter waves, cnoidal theory for sufficiently long waves, with the Ursell number often used as a regime guide. The Fourier approximation method is the preferred reference method when high accuracy is required over a broad range of depths and heights, especially for velocities, pressures, integral quantities or waves close to the boundary of the perturbation theories.

The nonlinear solver is therefore the preferred method when free-surface shape, velocities, accelerations, energy, radiation stress or near-breaking behavior matter. The wavelength-only kernels, symbolic formulas, Pade model, neural model and effective-variable model are appropriate for fast wavelength estimates only when their calibration range is respected.

---

13. Dedication

This work is dedicated to **Dr John D. Fenton** (1945-), Australian civil engineer, hydraulic engineer, and applied mathematician, in recognition of his distinguished and continuing scientific contributions to mathematical hydraulics, computational fluid mechanics, and nonlinear water-wave theory. Further information, publications, technical resources, and software by Dr Fenton are available on his official website: www.johndfenton.com.

---

14. References

1. **Fenton, J. D. (1999).** *Numerical methods for nonlinear waves.* In P. L.-F. Liu (ed.), *Advances in Coastal and Ocean Engineering*, Vol. 5, World Scientific, Singapore, pp. 241–324.
   Relevance: Review of numerical methods for nonlinear waves, including the Fourier approximation method for steadily propagating periodic waves and more general propagation methods.
   URL: Fenton99Liu-Numerical-methods-for-nonlinear-waves.pdf

2. **Fenton, J. D. (1999).** *The cnoidal theory of water waves.* In J. B. Herbich (ed.), *Developments in Offshore Engineering*, Gulf Publishing, Houston, pp. 275–337.

Relevance: Modern presentation of cnoidal theory for long finite-depth waves, including practical application, accuracy limits and numerical cnoidal ideas.
URL: Fenton99Cnoidal-The-cnoidal-theory-of-water-waves.pdf

3. **Fenton, J. D. & Kennedy, A. B. (1996).** *Fast methods for computing the shoaling of nonlinear waves.* In *Proceedings of the 25th International Conference on Coastal Engineering*, Vol. 1, Orlando, pp. 1130–1143.
Relevance: Nonlinear wave propagation and shoaling over varying bathymetry.
URL: Fenton96+Kennedy-Fast-methods-for-computing-the-shoaling-of-nonlinear-waves.pdf

4. **Fenton, J. D. (1995).** *A numerical cnoidal theory for steady water waves.* In *Proceedings of the 12th Australasian Coastal and Ocean Engineering Conference*, Melbourne, pp. 175–180.
Relevance: Numerical cnoidal formulation for finite-depth long-wave calculations.
URL: Fenton95-A-numerical-cnoidal-theory-for-steady-water-waves.pdf

5. **Townsend, M. & Fenton, J. D. (1995).** *Numerical comparisons of wave analysis methods.* In *Proceedings of the 12th Australasian Coastal and Ocean Engineering Conference*, Melbourne, pp. 169–173.
Relevance: Wave analysis and inversion from pressure measurements; conditioning and method comparison.
URL: Townsend95+Fenton-Numerical-comparisons-of-wave-analysis-methods.pdf

6. **Kennedy, A. B. & Fenton, J. D. (1995).** *Simulation of the propagation of surface gravity waves using local polynomial approximation.* In *Proceedings of the 12th Australasian Coastal and Ocean Engineering Conference*, Melbourne, pp. 287–292.
Relevance: Nonlinear wave propagation and shoaling over varying bathymetry.
URL: Kennedy95+Fenton-Simulation-of-the-propagation-of-surface-gravity-waves-using-local-polynomial-approximation.pdf

7. **Fenton, J. D. (1993).** *Simulating wave shoaling with boundary integral equations.* In *Proceedings of the 11th Australasian Conference on Coastal and Ocean Engineering*, Townsville, pp. 71–76.
Relevance: Boundary-integral formulation for nonlinear wave transformation, with singularity subtraction.
URL: Fenton93-Simulating-wave-shoaling-with-boundary-integral-equations.pdf

8. **Fenton, J. D. (1990).** *Nonlinear wave theories.* In B. Le Méhauté & D. M. Hanes (eds.), *The Sea: Ocean Engineering Science, Part A*, Vol. 9, Wiley, New York, pp. 3–25.
Relevance: Key source for current definitions, steady-wave governing equations, Stokes/cnoidal regime guidance, Fourier approximation methods and integral properties.
URL: Fenton90b-Nonlinear-wave-theories.pdf

9. **Drennan, W. M., Fenton, J. D. & Donelan, M. A. (1990).** *Numerical simulation of nonlinear wave groups.* In *Proceedings of the 11th Annual Conference of the Canadian Applied Mathematics Society*, Halifax.
Relevance: Relevant to nonlinear and free-surface wave modelling.
URL: Drennan90-Numerical-simulation-of-nonlinear-wave-groups.pdf

10. **Fenton, J. D. & McKee, W. D. (1990).** *On calculating the lengths of water waves. Coastal Engineering*, **14**, 499–513.

Relevance: Wavelength determination for nonlinear waves in finite depth.
URL: Fenton90c+McKee-On-calculating-the-lengths-of-water-waves.pdf

11. **Fenton, J. D. (1988).** *The numerical solution of steady water wave problems. Computers and Geosciences*, **14**, 357–368.
Relevance: Core practical program for steady periodic nonlinear waves; finite/deep water, wavelength/period input, Eulerian or Stokes current, Newton solution with numerical Jacobian.
URL: Fenton88-The-numerical-solution-of-steady-water-wave-problems.pdf

12. **Fenton, J. D. (1986).** *Polynomial approximation and water waves.* In *Proceedings of the 20th International Conference on Coastal Engineering*, Vol. 1, Taipei, pp. 193–207.
Relevance: Relevant to nonlinear and free-surface wave modelling.
URL: Fenton86-Polynomial-approximation-and-water-waves.pdf

13. **Fenton, J. D. (1985).** *A fifth-order Stokes theory for steady waves. Journal of Waterway, Port, Coastal, and Ocean Engineering*, **111**, 216–234.
Relevance: Closed-form fifth-order Stokes expansion for steady waves in finite depth.
URL: Fenton85d-A-fifth-order-Stokes-theory-for-steady-waves.pdf

14. **Fenton, J. D. (1983).** *On the application of steady wave theories.* In *Proceedings of the 6th Australasian Conference on Coastal and Ocean Engineering*, Christchurch, pp. 65–70.
Relevance: Guidance on applicability and limits of steady-wave theories in engineering.
URL: Fenton83-On-the-application-of-steady-wave-theories.pdf

15. **Fenton, J. D. & Rienecker, M. M. (1982).** *A Fourier method for solving nonlinear water wave problems. Journal of Fluid Mechanics*, **118**, 411–443.
Relevance: Time-dependent Fourier method for full nonlinear water-wave equations, used for solitary-wave interactions; relevant background but not the main steady-wave collocation solver.
URL: Fenton82c+Rienecker-A-Fourier-method-for-solving-nonlinear-water-wave-problems.pdf

16. **Schwartz, L. W. & Fenton, J. D. (1982).** *Strongly-nonlinear waves.* In M. Van Dyke, J. V. Wehausen & J. L. Lumley (eds.), *Annual Review of Fluid Mechanics*, **14**, 39–60.
Relevance: Fundamental properties and approximations for strongly nonlinear wave motion.
URL: Schwartz82-Strongly-nonlinear-waves.pdf

17. **Rienecker, M. M. & Fenton, J. D. (1981).** *A Fourier approximation method for steady water waves. Journal of Fluid Mechanics*, **104**, 119–137.
Relevance: Foundational Fourier approximation method for steadily progressing periodic waves; finite Fourier series, collocation of nonlinear free-surface conditions and Newton solution.
URL: Rienecker81+Fenton-A-Fourier-approximation-method-for-steady-water-waves.pdf

18. **Fenton, J. D. & Rienecker, M. M. (1980).** *Accurate numerical solutions for nonlinear waves.* In *Proceedings of the 17th International Conference on Coastal Engineering*, Sydney, pp. 50–69.
Relevance: Benchmark accurate numerical solutions for nonlinear wave profiles and kinematics.
URL: Fenton80+Rienecker-Accurate-numerical-solutions-for-nonlinear-waves.pdf

19. **Fenton, J. D. (1979).** *A high-order cnoidal wave theory. Journal of Fluid Mechanics*, **94**, 129–161.
Relevance: Cnoidal-wave theory for finite-depth long waves and associated numerical formulations.
URL: Fenton79-A-high-order-cnoidal-wave-theory.pdf

20. **Fenton, J. D. & Mills, D. A. (1976).** *Shoaling waves: numerical solution of exact equations.* In D. G. Provis & R. Radok (eds.), *Proceedings of the IUTAM Symposium on Waves on Water of Variable Depth*, Canberra, Springer-Verlag, pp. 93–100.
Relevance: Nonlinear wave propagation and shoaling over varying bathymetry.
URL: Fenton76+Mills-Shoaling-waves-Numerical-solution-of-exact-equations.pdf

21. **Fenton, J. D. (1972).** *A ninth-order solution for the solitary wave. Journal of Fluid Mechanics*, **53**, 257–271.
Relevance: High-order analytic and series solution for solitary waves.
URL: Fenton72-A-ninth-order-solution-for-the-solitary-wave.pdf